DEPARTMENT OF ENGINEERING MATHEMATICS

Reactive SDN for Securing ICS Environments

A Criticality- and Operation-Aware Response Framework (CARS)

Amit Kiran Deb - 2711250

A dissertation submitted to the University of Bristol in accordance with the requirements of the degree of Master of Science in the Faculty of Engineering.

Friday 14th August, 2026

Supervisor: Dr. Joseph Gardiner

Declaration

This dissertation is submitted to the University of Bristol in accordance with the requirements of the degree of MSc in the Faculty of Engineering. It has not been submitted for any other degree or diploma of any examining body. Except where specifically acknowledged, it is all the work of the Author.

Amit Kiran Deb - 2711250, Friday 14th August, 2026

Contents

1 Introduction **1**

2 Background **3**
2.1 Industrial control systems and their security priorities . . . 3
2.2 Software-defined networking and OpenFlow . . . 3
2.3 Reactive intrusion response with SDN in ICS . . . 4
2.4 Trusting the control plane and its forwarding integrity . . . 4
2.5 Stateful security in the data plane . . . 4
2.6 Asset criticality and consequence-based prioritisation . . . 4
2.7 Operation awareness and the false-data-injection threat . . . 5
2.8 Industrial control system testbeds . . . 5

3 Design and Implementation **7**
3.1 Threat and attacker model . . . 7
3.2 Testbed architecture . . . 8
3.3 CARS design overview . . . 9
3.4 The enforcement pipeline . . . 10
3.5 The criticality and operation model . . . 10
3.6 Reference implementation . . . 12
3.7 Deployment . . . 13

4 Critical Evaluation **21**
4.1 Method and metrics . . . 21
4.2 Decision accuracy and false positives . . . 21
4.3 Criticality-graded response . . . 23
4.4 Wire-level campaign: armed versus unprotected . . . 24
4.5 Cross-layer validation . . . 26
4.6 Reaction window and latency . . . 29
4.7 Anatomy of a defended attack: two pivots end to end . . . 31
4.8 Does the defence disturb the process? . . . 34
4.9 Process-layer defence in depth . . . 35
4.10 Comparison with related work . . . 36
4.11 Threats to validity . . . 36
4.12 Ethics . . . 39

5 Conclusion **41**
5.1 Contributions . . . 41
5.2 What the evaluation establishes, and what it does not . . . 41
5.3 Future work . . . 42

A Supporting Material **47**
A.1 Full OpenFlow tables . . . 47
A.2 Deployed decision logic . . . 49
A.3 Detection rules . . . 51
A.4 Runtime state . . . 52
A.5 Complete evaluation matrix . . . 52
A.6 Decision logs and events . . . 53

A.7 Packet-level and wire-level analysis 53
A.8 Scenario testing in full 54
A.9 Scripts and code 55
A.10 The CARS intelligence dashboard 61
A.11 Gap remediation evidence 62
A.12 Process-transparency and availability runs 64

List of Figures

3.1 The SDN-ICS testbed topology. Three OpenFlow switches under a single controller: Cell-1 and the gateway on Dell 1, and Cell-2 on Dell 3 reached through a NAT clone. Addresses, criticality tiers and ofports are read from the running fabric; the full port and binding map is in Appendix A. 8

3.2 The physical process cell. The Siemens S7-1212C PLC and its operator HMI are wired into the cell through the labelled HMI and PLC network ports, which face the Open vSwitch fabric on Dell 1. The green terminal blocks carry the PLC's 24 V digital inputs and relay outputs. Photographed on the testbed. 9

3.3 The fabric read from the switches with `ovs-vsctl show`: Cell-1 (`ovs1`, top) and Cell-2 (`ovs2`, bottom). Each bridge is connected to the controller at `tcp:10.10.10.1:6653` with `fail_mode: secure`, and carries its PLC and HMI on separate ports; `ovs1` also holds the patch to the gateway and `ovs2` the internal `cell2gw`. Captured live on 3 August 2026 (OVS 3.3.4); the arrows are annotations. 15

3.4 The hardware-in-the-loop. Factory IO supplies the level and button inputs and renders the valve and lamp outputs; PLC1 runs the control in OB30; every exchange crosses the CARS fabric. Addresses and mappings are from the deployed project. 15

3.5 The live process. Left: the Factory IO tank scene, with the level meter, the fill and discharge valves, the flow meter, and the Start, Stop and Reset station. Right: the operator HMI reading the tank level, here about 40 per cent, and the pump value. The scene runs on the Windows machine and exchanges its inputs and outputs with PLC1 over S7 across the fabric. 16

3.6 The Factory IO S7-1200 driver, bound to PLC1 at `192.168.2.10`. The inputs are the Start, Reset and Stop buttons at `%I10.0`–`%I10.2`, the running flag at `%I10.3`, and the level, flow and setpoint values at `%ID100`/`%ID104`/`%ID108`; the outputs are the lamps at `%Q0.0`–`%Q0.2`, the fill and discharge valves at `%QD100`/`%QD104`, and the setpoint and process value at `%QD108`/`%QD112`. Read from the deployed driver configuration; these match the OB30 tag table. 16

3.7 CARS in the fabric. All traffic to a protected asset crosses the SDN fabric, which the controller governs; the Factory IO process closes a hardware-in-the-loop with PLC1. . . . 17

3.8 The CARS OpenFlow pipeline, captured live on `ovs1` and `ovsgw`. A packet is admitted only if its identity binds at Table 0, its connection is permitted and tracked at Table 1, and it is then forwarded at Table 2. Proactive policy carries cookie `0x00a2`; reactive responses installed by the controller carry `0x00ca`. 17

3.9 The criticality read live from the controller with `curl $API/cars/criticality`, backing Table 3.1: the six tiered assets and the weight of each tier. Captured on 3 August 2026. . 17

3.10 The CARS operation decision. The source role and operation give a tier by first match in the rulebook; a sensitive operation on a critical asset is elevated to forbidden; the tier and the observed rate then select the graded response. 18

3.11 Operation classes recovered by deep packet inspection from the S7comm and Modbus function fields, as encoded in the deployed Snort rules. 18

3.12 The detection-to-response path: a mirrored packet is classified by Snort, the alert is forwarded to the controller, which decides against the rulebook and criticality and installs the flow rule that enforces the response. 18

3.13 Deployment. The controller on Dell 2 governs the switches on Dell 1 and Dell 3 over the OpenFlow control plane; the fabric, the Snort sensor and the supporting agents run together on Dell 1, and the engineering workstation with Factory IO is the live process. . 19

3.14 The controller status from `curl $API/cars/status`: the three switches connected (datapath identifiers `2`, `1`, `3`), identity and ARP guards enabled, and the running mean decide time. Captured on 3 August 2026. 19
3.15 The deployed controller running live on Dell 2. Each line records one decision: the source and destination, the protocol and operation, the verdict and response, the asset criticality, and the decide-and-enforce time. Captured from the controller console on 3 August 2026, confirming the engine runs as a service and logs every decision. 19

4.1 The reactive rule duration measured live for an identical forbidden control against targets of each criticality tier: 75, 60, 45 and 30 seconds for weights 3, 2, 1 and 0, following $30 + 15w$. Same actor and operation throughout; only the target's tier changes. 24
4.2 The path of a malicious S7 write in each state. Disarmed, the write crosses the fabric, reaches PLC1 and drives the tank to overflow. Armed, the reactive rule (`cookie 0x00ca`) drops it in the fabric, so it never reaches the PLC and the process is held. 25
4.3 Process devastation, disarmed. Left: the Factory IO tank driven full and over the top. Right: the operator panel at the same moment, the level cresting past the full mark at `+104.326` with the high-level alarm raised. The attacker holds the process out of its safe band and the operator cannot recover it. 26
4.4 The same armed isolation seen at the decision layer on the CARS dashboard: the compromised supervisory host `192.168.2.31 (scada)` issuing S7 CONTROL to the CRITICAL PLC1 is classed FORBIDDEN and met with an ENFORCED ISOLATE. This is the cross-layer counterpart to the datapath rule of Figure 4.5: the same event as a policy decision rather than an installed flow. 26
4.5 Armed enforcement at the datapath. The reactive source-quarantine rule installed on `ovs1` and `ovsgw` in response to the injection: `cookie=0xca`, `priority=110`, `nw_src=192.168.2.31`, `actions=drop`, `hard_timeout=75`. The non-zero packet counters are the attacker's own traffic being dropped; the rule self-expires after 75 s. 27
4.6 The same FDI injection from the compromised supervisory host `192.168.2.31`, run over a matched forty-second window with CARS disarmed then armed: malicious S7 writes reaching the PLC, 973 disarmed against 0 armed. Armed, the source is classified FORBIDDEN and isolated before its S7 session commits a write. Measured from the injection runs and the controller decision log. 27
4.7 The drop on the wire, armed, the S7 conduit filtered to the attacker (`s7comm && ip.src==192.168.2.31`). Top, at the Snort mirror: the write frames are seen (1348 at `6.380 s`, 1415 at `6.691 s`). Bottom, at the PLC port: only the first write (1374 at `6.380 s`) arrives; every later write is dropped in the fabric. 29
4.8 The malicious operation on the wire (frame 1348). CARS recovers the operation from the S7 parameter, `Function: Write Var (0x05)` to `Area: Outputs (Q) (0x82)`, an output actuation, and classifies it as CONTROL, forbidden on the CRITICAL PLC. . . . 30
4.9 Reaction window over one hundred autonomous trials, attack frame to enforcement on a single clock. Median 7.6 ms, 95th percentile 38.4 ms, 99th 67.9 ms, maximum 72.9 ms; every trial ended in an ISOLATE. The window is the detect-then-respond plumbing (Snort alert to installed flow); the upper tail reflects queueing at the single-threaded bridge under a heavier background alert load, not a fixed poll boundary. 31
4.10 The two pivots from one Kali. `eth2` routes north and source-NATs to the gateway identity `192.168.2.1`; `eth1` sits on the OT segment as `192.168.2.77`. Both are quarantined at the gateway switch before a single packet reaches PLC1, but on different `nw_src` identities. 32
4.11 The CARS decision pipeline and where each attack class terminates. The two pivots (non-conduit sources) are cut at Table 1; the allowlisted insider is delivered one packet, then caught by the DPI loop that recovers the S7 operation and installs the `0x00CA` isolate. . . 32
4.12 The reaction window placed against the attacker's packet cadence. The `0x00CA` isolate lands within the measured mean-time-to-mitigate (median 7.6 ms), while the attacker's TCP retransmits arrive ∼1 s apart, so the first SYN is seen once and every later packet meets the installed drop. 34

A.1 GUARD anti-spoof drop counters from `curl $API/cars/guard`, per binding and per switch (all zero, armed and quiescent). 52
A.2 The OpenFlow meter behind the THROTTLE response, from `ovs-ofctl dump-meters ovsgw`. 52

A.3 The Modbus operation on the wire, same conduit .31→.20. Left: Function Code: Read Holding Registers (3), permitted. Right: Function Code: Write Single Register (6) setting register 4 to 7, throttled. The verdict turns on this field. 53
A.4 Disarmed, the attacker's S7 Write Var (frame 452) reaches the PLC. With CARS armed the same write is dropped in the fabric (Figure 4.7). 54
A.5 The attacker's Setup frame (.31→.10, highlighted) amongst the legitimate .55 Factory IO loop traffic on the PLC port. CARS must pick the malicious operation out of a continuous stream of legitimate S7 reads and writes. 54
A.6 The sensor-spoof deception, disarmed. Left: the real Factory IO tank, filled. Right: the operator HMI at the same moment, reading zero. The reported process has been divorced from the real one. 55
A.7 Reconnaissance from Kali, armed. Top: the port scan of PLC1, every port filtered. Bottom: the reactive rule installed against the scanner, cookie=0xca, priority=110, nw_src=192.168.2.77, actions=drop, hard_timeout=75. 56
A.8 Identity spoofing at GUARD. After forging ARP as the trusted seam 192.168.2.31 from Kali, the anti-spoof counter dpid3 arp 192.168.2.31 registers the dropped frames (running total 10), every forged frame dropped at ingress with all other bindings untouched. The controller logged each as REFUSED (identity-spoof). 56
A.9 Reaction window (mean-time-to-mitigate) over one hundred autonomous trials, attack-frame to enforcement on a single clock: median 7.6 ms, mean 11.2 ms, standard deviation 11.8 ms, minimum 5.5 ms, maximum 72.9 ms, every trial ending in ISOLATE. 57
A.10 The CARS dashboard's discovered topology. Switches (ovsgw, ovs1, ovs2) in blue, the supervisory hosts (EWS, SCADA, the historians, CARS-Remediation) and PLCs discovered live with their roles and criticality rings, the CARS controller and Snort IDS, the modelled north side (Enterprise firewall, DMZ/jump, OT NAT) and the Cell-2 NAT path to PLC2. Unregistered hosts (the on-segment attacker seams 192.168.2.66 and 192.168.2.67) are flagged. Everything shown is polled live from the controller, not drawn by hand. 61

List of Tables

3.1 Asset criticality on the testbed, from the deployed controller (confirmed against the `/cars/criticality` interface, Figure 3.9). The weight scales the response: a block or isolate lasts $30 + 15w$ seconds, and escalation is faster at higher weight. 11

3.2 The rulebook, evaluated first match wins (rows with a shared tier grouped for presentation; the exact deployed order is in Appendix A). A SENSITIVE operation on a CRITICAL asset, outside a maintenance window, is elevated to FORBIDDEN. 12

4.1 The decision matrix, measured live from the deployed engine via `/cars/respond`. Hosts are `192.168.2.x` (last octet shown). Twenty-four scored cases: 13 TP, 11 TN, 0 FP, 0 FN; three grey cases show grading. Regenerated for this writeup; identical to the 3 August baseline. Full per-decision controller log in Appendix A.6. 22

4.2 Confusion summary. Accuracy $24/24 = 100\%$; false-positive rate $0/11 = 0\%$; false-negative rate $0/13 = 0\%$. 22

4.3 The campaign, armed against unprotected. Every *armed* outcome was captured live on 5 August, the external vectors from the Kali attacker `192.168.2.77` and the insider vectors from the compromised supervisory host `192.168.2.31`, corroborated by the controller decision log, the nmap and GUARD dumps, and the installed flows (Appendix A.8); the process-devastation contrast was re-captured fresh on the live tank (Section 4.4.1). The two *unprotected* reconnaissance and spoofing entries are the known flat-network baseline, an unenforced switch trivially serves a port scan and accepts a forged frame, and are given as reference rather than as a fresh measurement of our system. The legitimate Factory IO loop and historian telemetry were permitted throughout, so the campaign held zero false positives. 28

4.4 Cross-layer proof from the deep end-to-end trial: each response, with the Snort alert delta (the count of new lines appended to `/var/log/snort/alert` at that step, so a non-zero value confirms detection fired and `alert` marks a fired alert not separately counted), the controller tier, the installed OpenFlow rule, and the client outcome. The last row is the proactive default-deny with the detection bridge off, showing a drop with no Snort or controller involvement and therefore no reaction window. 28

4.5 The drop shown on the wire. Attacker S7 Write-Var frames counted from the pcaps at the two capture points. Armed, eight are seen by the detector but seven are dropped in the fabric and only one leaks through in the reaction window; disarmed, seven of eight reach the PLC. 29

4.6 Operation-awareness on the same conduit. The tier and response are the policy, correct for every operation, as the decision matrix of Section 4.2 establishes by driving each operation through the engine directly. The detection column records how reliably the Snort sensor recovers that operation from the wire. 30

4.7 The same attack, two positions, followed through nine stages. Enforcement is identical; the granularity of the quarantine is not. 33

5.1 Aims and objectives against outcomes. 42

A.1 Map from the report to the published repository (the project GitHub repository). 47

List of Listings

A.1 Cell-1 switch `ovs1`, tables 0 to 2. . . . 47
A.2 Gateway switch `ovsgw`, tables 0 to 2. . . . 48
A.3 Cell-2 switch `ovs2`, tables 0 to 2. . . . 49
A.4 The deployed `RULEBOOK` (first match wins). . . . 49
A.5 The A2 allowlist and default-deny (`a2_policy.json`). . . . 50
A.6 S7 and Modbus operation-class rules from `cars.rules`. . . . 51
A.7 The evaluation matrix, `cars_eval_matrix.csv`. . . . 52
A.8 Representative controller decisions across the campaign (extract of the decision log; times `08-05`). . . . 53
A.9 Modbus read and write on the same conduit; the function code sits at offset 7. . . . 53
A.10 The reactive isolate as an `OFPT_FLOW_MOD`, decoded from `of_control.pcap` (OpenFlow 1.3, tcp/6653). . . . 54
A.11 The controller decision path (`cars_engine.py`): first-match rulebook classification. . . . 55
A.12 Grey-zone elevation and criticality-scaled block duration. . . . 55
A.13 Criticality-graded, flood-aware response selection. . . . 56
A.14 Source quarantine: a self-healing drop on every switch (`REACTIVE_COOKIE = 0x00CA`). . . . 57
A.15 Table 0 identity binding and anti-spoof drop. . . . 57
A.16 Token gate on the control endpoints. . . . 57
A.17 Operation recovery from the Snort alert, forwarded to the controller. . . . 58
A.18 Flow identity and the drift model. . . . 58
A.19 Anomaly-triggered last-good restore over S7. . . . 58
A.20 Single-clock mean-time-to-mitigate measurement. . . . 58
A.21 The FDI overflow injection (writes to the S7 output area). . . . 58
A.22 Driving each case through the engine and scoring it against intent. . . . 58
A.23 Fire a controlled case, and heal the tier-scaled reactive rule between fires. . . . 59
A.24 Arm/disarm and the three-point capture used across the campaign. . . . 59
A.25 The three capture points and the control-plane vector. . . . 59
A.26 The four independent ground-truth layers per scenario. . . . 59
A.27 Two synchronised capture points: what the detector sees versus what reaches the PLC. . . . 59
A.28 Fire each operation on the same conduit and await the new decision. . . . 59
A.29 The output-area write (actuation) and the sensor-spoof variant. . . . 60
A.30 Crafting an arbitrary Modbus function code (FC at offset 7). . . . 60
A.31 The bang-bang control law, the dynamics, and the interference health metric. . . . 60
A.32 Stream reassembly, and the offset-to-relative rule change. . . . 60
A.33 Sending the Write-Var whole or split at byte N (0x05 pushed into the second segment). . . . 60
A.34 One authorised HMI stop; and pinning the level sensor high. . . . 60
A.35 The high-side envelope, rate and liveness checks (alarm-only). . . . 61
A.36 Gap A: before (fragmented write evades) and after (caught), from the controller audit. . . . 62
A.37 Gap A: `cars.conf` diff, stream reassembly added on ports 102 and 502. . . . 62
A.38 Gap A: `cars.rules` diff, S7 rules re-anchored PDU-relative (rev 1 to rev 2). . . . 62
A.39 Gap B: the flood backstop and the two guardian alarms. . . . 63
A.40 Gap B: the guardian alarm feed (`cars_guardian.jsonl`), stall record from the fresh run. 63
A.41 Gap C: transient (mostly missed) versus persistent (always caught) flow-integrity status, with the thirty-trial totals. . . . 63
A.42 Gap C remedy: event-driven monitor on the same transient test. . . . 63
A.43 Gap 2: shared-identity conduit BLOCK versus single-host source ISOLATE, deployed engine. 64
A.44 Gap D: silent-drop baseline, the redirect, and the decoy receiving the diverted traffic. . . . 64

A.45 Transparency: online-only level statistics per phase, from `interleaved.csv`. 64
A.46 Block-and-maintain under attack, then self-heal, from the armed run. 64
A.47 Reconnection jitter: the two planes contrasted. 65

Abstract

Automated intrusion response is rarely switched on in operational technology, and the reason is trust rather than detection. A reactive defence that blocks traffic can cut the very process it is meant to protect, and an operator will not arm a measure that might trip a running plant. This project addresses that barrier with CARS, a criticality- and operation-aware software-defined-networking intrusion-response system for industrial control systems. Instead of acting uniformly, CARS conditions its response on two properties that plain blocking ignores: the criticality of the asset under attack, and the industrial operation, recovered from the traffic by a deep-packet-inspection sensor, so that reading a value from a controller is treated differently from writing one. Its responses are graded across a seven-rung ladder, bounded in time, reversible, and recorded per decision.

CARS was built on a hardware testbed rather than a simulation. Two Siemens S7-1212C programmable logic controllers drive a live tank-level process through a software-defined fabric, with an operator panel and an engineering workstation present as in a small plant. The system is realised as a three-table OpenFlow pipeline: identity binding against spoofing, a stateful policy stage carrying the criticality- and operation-aware rulebook, and a learning switch, supported by a flow-integrity self-check and an authenticated control interface. Detection is delegated to the packet sensor, while the software-defined network layer makes the graded decision and enforces it as OpenFlow rules. Every capability was validated at the level of individual packets, protected against unprotected.

The central result is that the armed defence refused every illegitimate operation while cutting none of the legitimate process traffic: a decision accuracy of one hundred per cent at a false-positive rate of zero on the live process, with enforcement shown on the wire rather than inferred from a log, and a reaction window with a median of 7.6 milliseconds. A false-data injection that overflowed the tank when unprotected was severed on its first write when the defence was enabled. The evaluation carries its boundaries honestly, including the trusted-insider case that a network layer must not act on. The finding is that the trust barrier which keeps such defences switched off is not fundamental: a reactive network defence can be made criticality- and operation-aware, bounded and reversible enough for an operator to arm on a running process.

Supporting Technologies

The following third-party hardware and software were used to build and evaluate CARS. The contribution of this project is the CARS design, its controller engine, and the evaluation; the items below are the platform it was built on.

Hardware

- Three Dell workstations forming the testbed: Dell 1 runs the Open vSwitch fabric, the Snort sensor and the supporting services, and hosts the supervisory and attacker endpoints; Dell 2 runs the CARS controller; Dell 3 runs the second cell.
- Two Siemens S7-1200 programmable logic controllers driving the physical process. PLC1 is a 1212C, model 6ES7 212-1BE40-0XB0, firmware 4.2.3, confirmed by scanning it during evaluation.
- A Siemens KTP700 operator panel as the human-machine interface on the process cell.
- A hAP access point providing the isolated wired management network that carries the OpenFlow control plane, kept separate from the operational-technology data plane.
- A Windows PC acting as the engineering workstation, running TIA Portal and Factory IO.

Software

- TIA Portal, used to program the PLCs and the OB30 tank-control logic.
- Factory IO with its S7-1200 driver, providing the hardware-in-the-loop tank-level process wired to the live PLC.
- Open vSwitch, providing the software-defined data plane (the switches `ovsgw`, `ovs1` and `ovs2`) under OpenFlow 1.3 control.
- os-ken, the OpenFlow 1.3 controller framework hosting the CARS engine (`cars_engine.py`); the engine and the supporting tools are written in Python.
- Snort, used as the deep-packet-inspection sensor that recovers the industrial operation from S7 and Modbus traffic.
- python-snap7, the S7 client library used by the process model, the remediation agent and the attack clients.
- Linux network namespaces, used to isolate the supervisory endpoints (SCADA, Modbus operator, historian, remediation agent) on Dell 1, each bound to a switch port.
- Kali Linux, used as the external attacker virtual machine attached to the operational segment, together with nmap for reconnaissance and a set of custom Python clients for the S7 and Modbus write, false-data-injection and fragmentation tests.
- tcpdump, with Wireshark and tshark, used to capture and analyse traffic at the wire points for the packet-level evaluation.

Notation and Acronyms

The acronyms used in this report are listed below, followed by a note on the S7 addressing referred to in the design and evaluation chapters.

Acronym	Meaning
CARS	Criticality- and operation-Aware Response System (this project)
ICS	Industrial control system
OT	Operational technology
IT	Information technology
PLC	Programmable logic controller
HMI	Human-machine interface
EWS	Engineering workstation
SCADA	Supervisory control and data acquisition
SDN	Software-defined networking
OVS	Open vSwitch
DPI	Deep packet inspection
IDS	Intrusion detection system
FDI	False data injection
HIL	Hardware-in-the-loop
NAT	Network address translation
GUARD	The identity-binding, anti-spoof stage of the pipeline (Table 0)
MTTM	Time to mitigate (the reaction window from detection to enforcement)
REST	Representational state transfer (the controller's control interface)
TCP	Transmission control protocol
FC	Modbus function code
OB	Organisation block (PLC program unit)
DB	Data block (PLC memory area)
S7	Siemens S7 communication protocol
TP, TN, FP, FN	True positive, true negative, false positive, false negative

S7 addressing follows the Siemens convention used throughout: `%I` for process inputs and `%Q` for process outputs at the bit level, `%ID` and `%QD` for the corresponding double-word (32-bit) analogue channels, and `%M` for internal memory bits (merkers). For example, `%ID100` is the analogue level input from the tank sensor, `%QD100` and `%QD104` drive the fill and discharge valves, and `%M0.1` and `%M0.2` are the HMI start and stop bits.

Acknowledgements

I thank my supervisor, Dr. Joseph Gardiner, for his guidance and feedback throughout this project. I am grateful to the staff of the University of Bristol Cyber Security group for access to the laboratory and its equipment. I also thank my family for their support during the work.

Chapter 1

Introduction

Industrial control systems (ICS) run the physical processes behind electricity generation, water treatment, gas distribution and manufacturing. They differ from ordinary information technology in one respect that shapes every security decision: safety and the continuity of the process come first. The NIST guide to operational-technology security states that OT security objectives typically prioritise integrity and availability, followed by confidentiality, and must treat safety as an overarching priority [1]. A controller that stops driving its process can therefore cause a physical hazard rather than only a loss of data. A programmable logic controller (PLC) that loses contact with its sensors part way through a cycle may overfill a tank or trip a turbine. Any defensive measure placed in front of such a device therefore carries a risk of its own.

Software-defined networking (SDN) separates the control of a network from the switches that forward its traffic, so a central controller can install or withdraw forwarding rules as conditions change. This makes SDN an attractive base for reactive intrusion response in ICS: when a threat is detected, the controller can act on the offending traffic across the whole network in a coordinated way. The open question is what that action should be. A survey of SDN-based intrusion-response systems for ICS organises the field into three families of response strategy, of which the most direct, dynamic traffic filtering, acts by dropping packets or blocking flows [2]. None of these strategies conditions its response on the criticality of the device being protected. Cutting traffic to a safety-critical PLC in the middle of its control loop can be as harmful as the attack it is meant to stop. This is why automated response is still rarely enabled in operational technology. The barrier is not detection accuracy but trust: an operator will not switch on a defence that might trip the process itself.

The threats such a defence must answer are well documented and physical in their consequences. An attacker who reaches the control network can enumerate the PLCs, impersonate a trusted engineering station, or inject false sensor data so that the controller drives the process to an unsafe state while its own readings appear normal [3]. Because the SDN controller holds the forwarding policy for the whole network, a compromise of the control plane is itself a recognised attack, able to insert or remove rules silently [4]. A response system that reacts to these threats with a single blunt action inherits the trust problem described above; one that reacts without regard to which asset it is protecting cannot claim to be safe for the process.

This project addresses that gap with CARS, a criticality- and operation-aware SDN intrusion-response system for ICS. The central idea is to condition the response on two properties that uniform blocking ignores: the criticality of the asset being protected, and the nature of the operation being attempted. Criticality follows a consequence-based tiering drawn from established guidance, in which each asset is rated by the physical and operational consequence of its compromise [5, 6]. The operation is recovered from the traffic itself, so that reading a value from a PLC is treated differently from writing one, and an engineering download is treated differently again. Responses are graded rather than binary, and each is designed to be bounded in time, reversible, and to leave an audit record. The intent is a defence an operator can enable, because it will refuse an unsafe action on a critical asset before it will ever interrupt the process that keeps that asset running.

The work is grounded in a hardware testbed rather than a simulation. Two real Siemens S7-1212C PLCs (model 6ES7 212-1BE40-0XB0, firmware 4.2.3, confirmed by scanning the device during evaluation) drive a live tank-level process through the SDN fabric, with an operator panel and an engineering workstation present as they would be in a small plant. The physical process allows a response to be judged against its real consequence: whether the tank stays within its safe band, or overflows. The cen-

tral challenges the project had to resolve were to define a criticality and operation model that is precise enough to enforce, to realise it as an OpenFlow pipeline that does not add meaningful latency to the control path, and to show, at the level of individual packets, both that the defence stops the attacks and that it never blocks the legitimate process.

Aims, objectives and achievements

The aim of the project was to design, build and evaluate an SDN intrusion-response system for ICS whose response is conditioned on asset criticality and operation, so that automated response can be enabled without risking the physical process. This was pursued through the following objectives:

- Build a representative hardware testbed comprising an SDN fabric, real PLCs, an operator panel and a live physical process.
- Design a response model that grades enforcement by asset criticality and operation class, and that is bounded, reversible and evidence-generating.
- Implement the model as an OpenFlow controller with a layered pipeline, together with the integrity checks needed to remain trustworthy under control-plane compromise.
- Integrate a hardware-in-the-loop process so that responses can be tested against real physical consequences.
- Evaluate decision accuracy and false positives, and validate each capability at the packet level against realistic attacks, comparing protected with unprotected operation.

The main achievements of the project, each evidenced in the chapters that follow, are:

- A working three-node SDN-ICS testbed with two real S7-1212C PLCs and a live hardware-in-the-loop tank process driven through the controlled fabric.
- An implementation of CARS as a three-table OpenFlow pipeline: identity binding against spoofing, a stateful policy stage carrying a criticality- and operation-aware rulebook, and a graded seven-level response ladder, supported by a flow-integrity checker and an authenticated control interface.
- A decision evaluation across the range of role, operation class and asset criticality in which every legitimate operation was permitted and every illegitimate one was refused, with no false positives against the live process traffic.
- A packet-level validation of each capability, protected against unprotected, spanning reconnaissance, identity spoofing, out-of-state traffic, control-plane and flow tampering, operation-aware commands, and a false-data injection that overflowed the tank when unprotected yet was severed on its first write when the defence was enabled.

Chapter 2

Background

This chapter sets out the technical basis the project depends on and the work it builds on. It moves from the security priorities of industrial control systems, through the use of software-defined networking to react to attacks in these environments, to the more specific problems of trusting the control plane, keeping security decisions stateful in the data plane, recognising the operation an attacker is attempting, and grading a response by the consequence of acting on an asset. The intent is that a reader who is technically competent but new to the topic can follow, and fairly assess, the design and evaluation presented in later chapters.

2.1 Industrial control systems and their security priorities

Industrial control systems (ICS) supervise and drive physical processes in sectors such as electricity, water, gas and manufacturing. A programmable logic controller (PLC) reads sensors, runs a control program and drives actuators such as pumps and valves, often within a cycle measured in milliseconds. This coupling to the physical world changes what security has to protect. The NIST guide to operational-technology security states that OT security objectives typically prioritise integrity and availability, followed by confidentiality, and must treat safety as an overarching priority [1]. Etxezarreta et al. give the same priority from the network side, noting in their survey that availability is a priority in industrial control systems, whose concerns include fault tolerance and human safety, so that an unexpected system outage is not acceptable [2]. A controller that is prevented from driving its process can therefore cause a physical hazard rather than only a disclosure of data. A defence for such an environment must therefore be judged not only on whether it stops an attack, but on whether the act of defending disturbs the process.

A brief note on how such a controller is programmed helps the reader who comes from networking rather than automation. A PLC runs on a repeating scan: each cycle it copies the field inputs into a process image, executes its logic, and writes the outputs. That logic lives in organisation blocks that the runtime calls automatically, such as OB30, the cyclic block that holds this project's tank control. Memory is addressed by area: `%I` and `%Q` are the input and output images at the bit level, `%ID` and `%QD` the 32-bit analogue channels, and `%M` internal memory bits. So a level sensor might arrive at `%ID100`, a valve be driven from `%QD100`, and an operator button be held in a `%M` bit. These conventions recur throughout the design and evaluation chapters.

2.2 Software-defined networking and OpenFlow

Software-defined networking (SDN) separates the decision about how traffic is forwarded, the control plane, from the switches that carry it, the data plane. A central controller holds a global view of the network and installs forwarding rules into the switches, most commonly through the OpenFlow protocol, so the behaviour of the network can be changed in software rather than by reconfiguring each device by hand [7]. For intrusion response this programmability is attractive, because when a threat is detected the controller can add or remove rules across the network in a coordinated way [2]. The same centralisation is also a weakness, discussed in Section 2.4, since the controller becomes a single point through which the whole forwarding policy can be altered.

2.3 Reactive intrusion response with SDN in ICS

Etxezarreta et al. survey how SDN has been used to build intrusion-response strategies for ICS and organise the field into a taxonomy of three families [2]. The first is dynamic traffic filtering, in which a detection technique such as deep packet inspection, signature matching, anomaly detection or machine learning identifies malicious traffic and the controller drops the offending packets or blocks the flow so that it does not reach the target device. The survey's own example of this family detects payload alteration on the path between a SCADA server and a PLC and drops the malicious control packets before they arrive [2]. The second family is network survivability, which uses the global view of the network to detect a failure or attack and reconfigure paths so that operation continues. The third is cyber deception, which includes moving-target defence, where addresses or paths are randomised to raise the attacker's uncertainty, and honeypots.

Two observations follow that motivate this project. First, the direct response actions in these strategies are to drop or to block, that is, to stop traffic reaching a device. Second, and more important, none of the three families conditions its response on the consequence of acting on the particular asset being protected. A blocked flow to a rogue scanner and a blocked flow to a PLC in the middle of a control loop are treated in the same way, even though the second can be as harmful as the attack it answers. This is the practical reason automated response is rarely enabled in operational technology: the obstacle is not the accuracy of detection but trust that the response will not itself trip the process. CARS addresses this gap by making the response conditional on the criticality of the protected asset and on the class of operation attempted, rather than applying a single uniform action.

2.4 Trusting the control plane and its forwarding integrity

Because the SDN controller sets the forwarding policy for the whole network, its compromise is a distinct and serious threat. Gardiner et al. show that the centralisation of control in SDN creates a single point of failure, and demonstrate controller-in-the-middle attacks in which a compromised or interposed controller manipulates the network in an ICS setting [4]. A response system that trusts its own control plane without question cannot claim to be safe against this class of attack.

The complementary defence is to verify the forwarding state itself. Melis et al. present a toolkit, built as plug-ins for the ONOS controller, that lets an administrator express security policies for an industrial SDN and check them formally against the installed rules. Their checker detects compromised forwarding caused by bogus injected flow-rules, inner loops and black-holes, which they note are hard to find through ordinary network scans, as well as the replacement or removal of legitimate rules [8]. The flow-integrity checker in CARS follows this line of work: it holds a trusted baseline of the security policy and reports any drift from it, so that tampering with the forwarding rules is caught even when the controller itself is the source of the change.

2.5 Stateful security in the data plane

Early SDN security work observed that pushing every decision to the controller is itself exploitable. Shin et al. introduce AVANT-GUARD, which extends the OpenFlow data plane with two mechanisms. Connection migration has the switch complete the TCP handshake and involve the controller only once a connection is established, which sharply reduces the traffic sent to the control plane during scanning or denial-of-service attacks. Actuating triggers allow conditional actions to be taken within the data plane in response to network conditions [9]. The relevant principle for this project is that security decisions can carry connection state in the data plane rather than treating each packet in isolation. The stateful stage of the CARS pipeline follows that principle: it tracks connection state so that only properly established flows are admitted, and out-of-state packets that a stateless rule would pass are refused.

2.6 Asset criticality and consequence-based prioritisation

The idea that a response should depend on the value of what is being protected is the basis of consequence-driven security engineering for control systems. The Idaho National Laboratory's Consequence-driven Cyber-informed Engineering examines high-consequence risk in critical infrastructure by first identifying the most damaging events an adversary could cause, then the key devices and components that enable them, the plausible cyber attack paths to those devices, and concrete mitigations and tripwires [5].

Operational guidance from CISA and international partners complements this at the inventory level. It recommends classifying operational-technology assets both by criticality, where a critical asset is one whose failure or compromise would have the most significant impact, and by function, such as control, monitoring, engineering or management [6]. CARS applies both ideas. Each protected asset is assigned a criticality tier according to the consequence of its compromise, and that tier, together with the class of operation, determines both the decision and the strength and duration of the response. The specific tiering used in this project is defined in Chapter 3.

2.7 Operation awareness and the false-data-injection threat

Traffic filtering in ICS can act on more than the source and destination of a flow. Because industrial protocols carry the operation being requested, deep packet inspection allows a defence to distinguish, for example, a read of a value from a write of one, and to treat an engineering download differently again; the survey lists deep packet inspection among the detection techniques used in the filtering family [2]. This matters because some of the most damaging ICS attacks are not floods but precise manipulations of control or sensor data. In a false-data-injection attack the adversary alters the values the controller reads or writes so that the process is driven to an unsafe state while its own readings appear normal; Etxezarreta et al. identify false-data injection as one of the main threats to ICS and propose a replica-based moving-target defence against it [3]. Reconnaissance is the usual first stage of such campaigns, and defending against it in ICS has itself been studied as a moving-target-defence problem [10]. CARS recovers the operation from the traffic and uses it in the decision, so that a read from a PLC and a write to it are not treated alike.

2.8 Industrial control system testbeds

Evaluating ICS security work requires a realistic environment, and building one is itself a recognised research problem. Antonioli and Tippenhauer present MiniCPS, a toolkit built on the Mininet emulator for reproducible security research on cyber-physical system networks, supporting simulated devices and industrial protocols [11]. Gardiner et al. observe that ICS security work often cannot be evaluated on real systems for safety and operational reasons, and distil design principles and lessons from building ICS security testbeds of their own [12]. This project takes the higher-fidelity option. Rather than a pure emulation, it uses a hardware testbed with two real Siemens PLCs driving a live physical process through the SDN fabric, so that a response can be judged against its actual effect on the process. The design of that testbed is described in Chapter 3.

Chapter 3

Design and Implementation

This chapter describes the design of CARS and its realisation on the testbed. It opens with the threat and attacker model that the design answers, then presents the testbed architecture, the layered enforcement pipeline, the criticality and operation model at its centre, and the supporting components, and closes with how these are deployed across the physical nodes. The chapter is arranged from the threats inward, so that each mechanism can be read against the specific capability it counters.

3.1 Threat and attacker model

3.1.1 Assumptions and capabilities

In keeping with the setting of the project, the attacker is assumed to have already reached the operational-technology network, whether by moving laterally from an enterprise network or by direct connection, which is the starting point taken by related work on defending ICS against reconnaissance [10]. Three attacker positions are considered, and all three are exercised in the evaluation. The first is an unregistered external host that is present on the OT segment but holds no legitimate role, represented in the testbed by a Kali Linux machine. The second is a trusted device that has been compromised, so that its legitimate network identity is used to issue illegitimate commands; this is represented by a compromised supervisory host. The third is a compromised control plane, in which the SDN controller itself, or a party interposed on its channel, manipulates the forwarding rules, which is the threat identified by Gardiner et al. for software-defined ICS networks [4]. From its position the attacker can send arbitrary traffic, including forged and out-of-state packets, and can spoof the network-layer and link-layer identity of other hosts.

3.1.2 Attacker goals

The attacker's objective is to disrupt or seize control of the physical process, or to gather the information needed to do so. The goals exercised in the evaluation, in the order an intrusion tends to follow, are the following.

- **Reconnaissance.** Discover the hosts on the segment and identify the PLCs, including model and firmware, to select an exploit [10].
- **Identity spoofing.** Impersonate a trusted host at the network or link layer to obtain the access that the identity carries.
- **State manipulation.** Inject forged or out-of-state traffic to slip past a stateless control or to disturb an established session.
- **Control-plane and flow tampering.** Insert, remove or rewrite forwarding rules, or attempt to disarm the defence, from a compromised control plane.
- **Unauthorised control.** Issue an operation that a given role should not perform on a given asset, such as writing an actuator or downloading a program to a safety-critical PLC.
- **False-data injection.** Falsify sensor or control values so that the PLC drives the process to an unsafe state while its own readings appear normal [3].

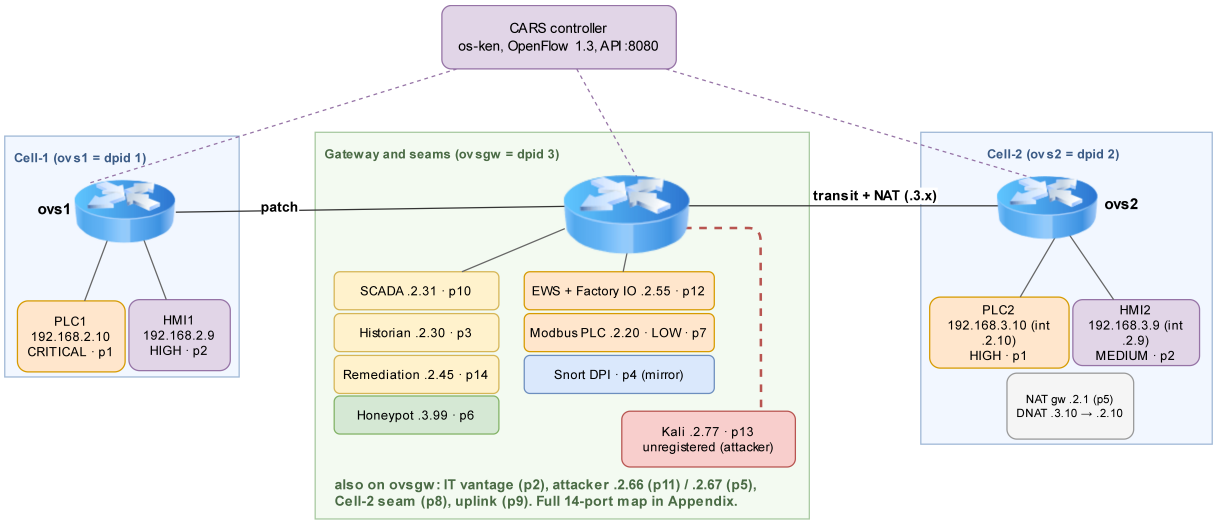

Figure 3.1: The SDN-ICS testbed topology. Three OpenFlow switches under a single controller: Cell-1 and the gateway on Dell 1, and Cell-2 on Dell 3 reached through a NAT clone. Addresses, criticality tiers and ofports are read from the running fabric; the full port and binding map is in Appendix A.

3.1.3 Attacker knowledge and scope

Attacker knowledge depends on position. The external host begins with no knowledge of the roles or addresses on the segment and must discover them, whereas the compromised trusted host already holds a legitimate identity and knows the conduits it is permitted to use. The model is bounded deliberately. Physical attacks on the devices, supply-chain compromise, and attacks on authentication mechanisms are out of scope, since the focus is network-borne attacks on the process once the perimeter has been breached, following the same scoping as related ICS reconnaissance work [10]. Two residual boundaries are carried honestly through the evaluation. An endpoint that is fully compromised while acting only within the operations its role is permitted is inside the trust the design grants that role. A fully subverted controller that knows the defence's own internal markings is outside what a self-checking control plane can detect. Both are examined in Chapter 4.

3.2 Testbed architecture

The work is built on a hardware testbed rather than an emulation, so that a response can be judged against its effect on a real process. The testbed comprises four nodes and two real PLCs, arranged as the two-cell topology of Figure 3.1. The physical process cell, with its PLC and operator HMI, is shown in Figure 3.2.

3.2.1 Nodes and devices

Four machines make up the testbed. Dell 2 runs the CARS controller as an os-ken OpenFlow 1.3 application. Dell 1 runs the Open vSwitch fabric, the Snort sensor, and the supporting services, and hosts the simulated supervisory and attacker endpoints in Linux network namespaces. Dell 3 runs Cell-2. A Windows machine runs TIA Portal and Factory IO and acts as the engineering workstation. The two PLCs are Siemens S7-1200 units; PLC1 is a 1212C, model 6ES7 212-1BE40-0XB0 running firmware 4.2.3, confirmed by scanning it during evaluation. An operator HMI is present on each cell. The supervisory hosts, the SCADA or Modbus operator, the historian, the remediation agent and the Modbus unit, are realised as namespaces on Dell 1, each bound to a switch port, and the external attacker is a Kali Linux virtual machine on Dell 1 that is attached to the OT segment and holds no registered role.

3.2.2 The two cells

The fabric is divided into two process cells and a gateway, each an OpenFlow switch under the one controller. Cell-1, on switch `ovs1`, holds PLC1, which controls the critical tank, and its operator HMI. Cell-2, on switch `ovs2` on Dell 3, holds PLC2 and its HMI, reached through a network-address translation that

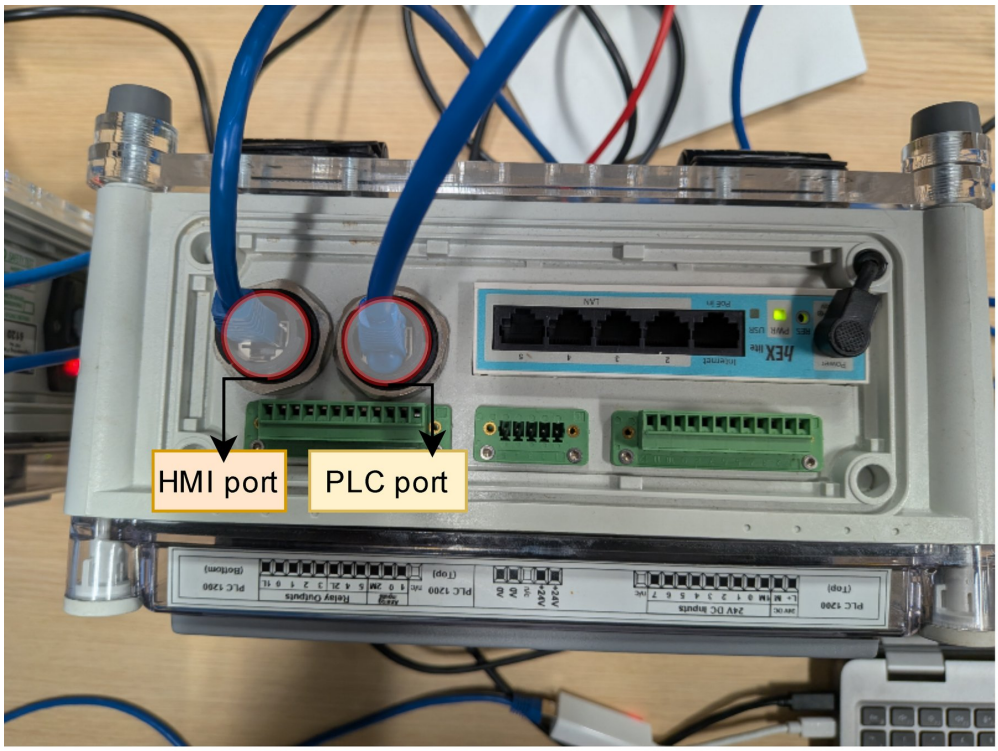

Figure 3.2: The physical process cell. The Siemens S7-1212C PLC and its operator HMI are wired into the cell through the labelled HMI and PLC network ports, which face the Open vSwitch fabric on Dell 1. The green terminal blocks carry the PLC's 24 V digital inputs and relay outputs. Photographed on the testbed.

presents their internal addresses `192.168.2.10` and `192.168.2.9` as `192.168.3.10` and `192.168.3.9` to the rest of the fabric; the translation lets the second cell reuse the Cell-1 addressing plan while remaining separately reachable. The gateway switch, `ovsgw`, carries the supervisory seams, the engineering and Factory IO host, the Snort mirror, the honeypot decoy, and the attacker vantage. The two cells and the gateway are joined by a patch link and a translated transit, so that all traffic to a protected asset crosses the CARS fabric and is subject to its policy. The fabric read directly from the switches with `ovs-vsctl show` is given in Figure 3.3: each bridge holds its OpenFlow connection to the controller, runs in secure fail mode, and presents the PLC and HMI on separate ports.

3.2.3 The physical process

Cell-1 runs a live tank-level process, provided by Factory IO on the Windows machine and controlled by PLC1 over S7 across the fabric, as shown in Figure 3.4. Factory IO supplies the level from a sensor and the state of the Start, Reset and Stop buttons as PLC inputs, and renders the fill valve, the discharge valve and the indicator lamps from the PLC outputs. The control runs in a cyclic block, OB30, which scales the raw sensor to a level of zero to one hundred, latches a running state from the operator buttons, and holds the level in a band by opening the fill valve below thirty and closing it above seventy against a constant discharge. The level that the PLC computes, held in a data block, is the process value that CARS protects; driving it out of its band, or blinding the PLC to it, is the physical harm the evaluation measures. The running scene and the operator HMI are shown in Figure 3.5, and the Factory IO driver that binds the scene to PLC1 in Figure 3.6.

3.3 CARS design overview

CARS is placed as the enforcement point in the SDN fabric, between the operational hosts and the assets it protects, as shown in Figure 3.7. Its design is layered. A proactive layer, always in force, binds each host's identity and admits only the connections on a short allowlist, so an unregistered host is refused before any operation is considered. A reactive layer recovers the operation from the traffic by deep

packet inspection, decides its tier from the source role, the operation and the criticality of the target, and applies a graded response. A self-checking layer verifies the forwarding state against a trusted baseline and authenticates the control interface, so that a compromise of the control plane is caught rather than trusted.

Two separations shape the design. The controller, which decides, is separate from the switches, which enforce, the standard software-defined split that lets one policy govern the whole fabric. The decision, which is a tier, is separate from the response, which is an action, so that a single judgement drives a graded ladder rather than one blunt action. Every response is bounded in time, reversible, and leaves an audit record, which is what allows an operator to enable it. The remaining sections detail each layer: the pipeline that enforces the policy (Section 3.4), the criticality and operation model that decides (Section 3.5), and the reference implementation of the detection, the responses and the self-checks (Section 3.6).

3.4 The enforcement pipeline

CARS enforces its policy in the data plane as a pipeline of three OpenFlow tables, installed on every switch in the fabric so that the same rules apply in each cell. A packet reaches its destination only if it passes all three stages; failing any stage drops or diverts it. The pipeline is shown in Figure 3.8. The flow rules quoted below are read from the running switches rather than from the source, and the full tables are reproduced in Appendix A, so that the description matches the deployed system exactly.

Table 0 performs identity binding, the defence against spoofing. Each protected host is bound to the switch port, MAC address and IP address from which it is permitted to speak. A packet whose source matches its binding, at priority 200 for both IP and ARP, is passed to Table 1; traffic arriving on an inter-bridge link, which has already been checked at the far switch, is passed at priority 150. A packet that claims a protected address but arrives on any other port matches no binding and falls to a priority-100 rule that drops it, for both IP and ARP. Traffic that claims no protected address is passed on at priority 50. The effect is that an attacker cannot borrow the network identity of a trusted host: a forged source is dropped at ingress before it reaches the policy stage.

Table 1 is the stateful policy stage, and it carries connection state in the data plane in the manner introduced by AvantGuard [9]. An untracked packet is first submitted to connection tracking at priority 90 and re-enters the table. A packet belonging to an already established connection is admitted at priority 85, which is what allows a protected device such as the HMI to receive the replies to connections it began while remaining shielded from new inbound connections. New connections are then judged against the proactive policy. A small allowlist of conduits, each naming a source, a destination and a service port, is permitted and committed to the connection table at priority 80; the deployed policy holds nine such conduits, including the operator and engineering paths to PLC1 and the cross-cell collector path to PLC2. A new connection to a protected asset that matches no conduit is dropped at priority 55, the default-deny that closes an asset to everything except its named conduits, and a new connection to any other asset is committed and passed at priority 10. The reactive responses described in Section 3.5 are installed into this same table by the controller at higher priorities, so that an isolate or a block takes precedence over the proactive rules for as long as an attack persists.

Table 2 forwards. It floods broadcast traffic, sends a packet whose source and destination MAC pair has been learned to the correct port, and refers an unknown destination to the controller so that the source can be learned. Separating the three concerns, identity at Table 0, connection state and policy at Table 1, and forwarding at Table 2, means each is enforced on its own, and a packet cannot reach forwarding by evading an earlier stage.

The proactive rules carry the cookie `0x00a2` and the reactive rules carry `0x00ca`. This separation is used by the flow-integrity checker of Section 3.6 to tell the fixed policy from the responses installed at run time.

3.5 The criticality and operation model

CARS makes two kinds of decision. At the connection level a proactive policy fixes which parties may open a connection to which asset at all. At the operation level each permitted exchange is judged by the role of its source, the operation it carries, and the criticality of the asset it targets. This section defines the model; the pipeline of Section 3.4 enforces it. Every value quoted is taken from the deployed controller.

Asset	Address	Role	Criticality	Weight
PLC1	192.168.2.10	plc	CRITICAL	3
PLC2	192.168.3.10	plc	HIGH	2
HMI1	192.168.2.9	hmi	HIGH	2
HMI2	192.168.3.9	hmi	MEDIUM	1
Historian	192.168.2.30	historian	MEDIUM	1
Modbus unit	192.168.2.20	plc	LOW	0

Table 3.1: Asset criticality on the testbed, from the deployed controller (confirmed against the `/cars/criticality` interface, Figure 3.9). The weight scales the response: a block or isolate lasts $30 + 15w$ seconds, and escalation is faster at higher weight.

3.5.1 Asset criticality

Every protected asset is assigned a criticality tier by the consequence of its compromise, following the consequence-based approach set out in Section 2.6 [5, 6]. Table 3.1 lists the tiers on the testbed. Each tier carries a weight from three down to zero, and the weight does two things in the response. It sets the duration of a block or an isolate, computed as thirty seconds plus fifteen for each unit of weight, so a critical asset holds an attacker out for seventy-five seconds against thirty for a low one, and it brings escalation forward, so a higher-consequence asset reaches a cut sooner. Criticality therefore shapes not only whether CARS acts but how hard and for how long.

3.5.2 Roles and operations

Each host has a role in the registry: *plc*, *hmi*, *historian*, *scada*, *ews* (engineering workstation), *remediation*, or *gateway*. A host with no registry entry, such as the external attacker, is *unknown*. The operation carried by a packet is recovered from the industrial protocol by deep packet inspection (Section 3.6): a read of a value, a write of one, a control action on the process image, a diagnostic request, a program download, or a malformed request, together with the plain connection setup. Treating the operation as a first-class input is what lets CARS permit an engineer to read a PLC while refusing that same engineer a program download.

3.5.3 The rulebook and criticality elevation

The tier of an operation is decided by a rulebook of (source role, destination role, operation, tier) tuples, evaluated first match wins. Table 3.2 gives the rules; the full deployed ordering is reproduced in Appendix A. The operator loop between HMI and PLC is tier CRITICAL and is never enforced against, a safety invariant so that the defence can never cut the loop that keeps the process safe. The dangerous operations are FORBIDDEN, subject to a deliberate ordering. A diagnostic, a program download or a malformed request is refused to a PLC or HMI from every source. A control write is refused from every source as well, except the two that are pre-authorised for it and whose rows are placed before the block, the engineering workstation and the remediation agent, so their legitimate actuation of the PLC classifies as OPERATIONAL rather than FORBIDDEN. Routine reads from the supervisory roles are OPERATIONAL, and an actuating write from a supervisory, historian or scada role is SENSITIVE. On top of the first match, one rule adjusts the tier by criticality: a SENSITIVE operation aimed at a CRITICAL asset, outside an authorised maintenance window, is elevated to FORBIDDEN, because nothing but the control loop and an authorised engineer may actuate the safety-critical process. The decision is shown in Figure 3.10. The resulting tier and the observed rate then select a graded response, which is described in Section 3.6.

3.5.4 Proactive policy

Alongside the per-operation decision, a proactive policy fixes which connections may exist at all, independently of any live detection. It is a short allowlist of conduits, each a source, a destination and a service port, together with a default-deny on the protected assets. The deployed policy, loaded from a run-time file and installed as the priority-80 and priority-55 flows of Section 3.4, holds nine conduits: the operator, engineering, remediation and historian paths to PLC1; the operator and historian paths to the Modbus unit; the engineering path to HMI1; and the Cell-2 collector path to PLC2. Every other new connection to PLC1, HMI1 or the Modbus unit is denied. An unregistered host is therefore refused at

Source role	Destination	Operation	Tier
hmi, plc (the loop)	plc, hmi	any	CRITICAL
remediation	plc	any	OPERATIONAL
ews	plc	read, write, control	OPERATIONAL
any	plc, hmi	control, diag, program, illegal	FORBIDDEN
ews	hmi	any	SENSITIVE
historian, scada, supervisory	plc, hmi	write	SENSITIVE
historian, scada, supervisory	plc, hmi	any	OPERATIONAL
any	any	any	FORBIDDEN

Table 3.2: The rulebook, evaluated first match wins (rows with a shared tier grouped for presentation; the exact deployed order is in Appendix A). A SENSITIVE operation on a CRITICAL asset, outside a maintenance window, is elevated to FORBIDDEN.

the connection level before any operation is considered, while the rulebook governs what the permitted parties may then do.

3.6 Reference implementation

The design of the preceding sections is realised on the testbed as an os-ken OpenFlow 1.3 controller on Dell 2, an Open vSwitch fabric and a Snort sensor on Dell 1, and the pipeline of Section 3.4. This section describes how each mechanism is built. The values quoted are read from the running system.

3.6.1 Identity binding and connection state

Table 0 realises the anti-spoof binding as OpenFlow rules. Each protected host has a rule, at priority 200, that matches its ingress port, source MAC and source address, for both IP and ARP, and passes the packet on; a companion priority-100 rule drops any packet that claims the same address from any other port. The bindings are pinned to fixed ofports, so that re-cabling cannot silently move a host and break its binding, a fault observed and corrected during development. Table 1 carries connection state using the switch's connection-tracking action: an untracked packet is submitted to conntrack and re-enters the table, an established connection is admitted, and only a handshaked new connection that matches a conduit is committed. Because state is held in the data plane rather than at the controller, an out-of-state packet that a stateless rule would pass is refused without a round trip to the control plane, in the manner of AvantGuard [9]. The full flow tables dumped from all three switches are reproduced in Appendix A.1, the deployed decision logic in Appendix A.2, the Snort detection rules in Appendix A.3, and a snapshot of the live runtime state in Appendix A.4.

3.6.2 Operation classification

The operation carried by a packet is recovered by deep packet inspection. A Snort sensor reads a mirror of the gateway switch and matches the function field of each industrial request. For S7comm to a PLC on port 102, the function byte that follows the protocol header distinguishes a read (`0x04`), a write or control action (`0x05`), and the diagnostic control and stop functions (`0x28` and `0x29`). For Modbus to the simulated unit on port 502, the function code distinguishes a register read (`0x03`), coil and register writes (`0x05`, `0x06`, `0x0F`, `0x10`), a diagnostic (`0x08`) and the encapsulated-interface program function (`0x2B`); any function code above the defined range is flagged as malformed. Figure 3.11 summarises the mapping. Each match raises an alert that names the operation, which is what allows the decision of Section 3.5 to treat the same connection differently according to what it carries.

3.6.3 From detection to response

Detection and response are joined by the chain shown in Figure 3.12. Snort writes each alert to a file; a bridge process follows the file and forwards each alert, with its source, destination and recovered operation, to the controller's `/cars/respond` interface; the controller classifies the operation against the rulebook and criticality, selects a response, and installs the corresponding flow rule. The controller's own decision time is 0.024 ms on average over the run captured here, so the path from a packet on the wire to an installed rule completes in the low milliseconds.

3.6.4 The response ladder

The controller selects one of seven graded responses and realises it in OpenFlow. ALLOW and MONITOR admit the traffic and record it. THROTTLE attaches a rate-limiting meter to the conduit. DEFLECT rewrites the destination to a honeypot and installs the forward and reverse rules for it. ISOLATE drops all traffic from the offending source at priority 110, and BLOCK drops the single offending conduit at priority 100. REFUSE is reserved for the safety-critical loop and only mirrors and alerts, never enforcing, so that the operator's control of the process can never be cut by the defence. Every reactive rule carries the cookie `0x00ca` and a criticality-scaled timeout of $30 + 15w$ seconds, so it expires and self-heals unless the attack persists. The thirty-second base is set to outlast an automated reconnect: a default TCP retransmission sequence spans roughly thirty-one seconds, so a thirty-second quarantine survives a full reconnect attempt rather than releasing the source mid-retry, while staying short enough that a mistaken block on a legitimate source clears in well under a minute. The fifteen-second step per weight makes the hold proportional to consequence, producing the 30, 45, 60 and 75 second ladder across the four tiers, measured in Section 4.3.

3.6.5 Forwarding integrity and control authentication

Two mechanisms protect CARS against a compromised control plane, the threat identified by Gardiner et al. [4]. A flow-integrity checker holds a trusted baseline of the proactive policy, distinguished by its cookie `0x00a2`, and compares the live tables against it every ten seconds; a rule that is missing, added or altered, other than the reactive rules under cookie `0x00ca`, is reported as drift, catching bogus injected rules, black-holes and removals in the manner of the policy checker of Melis et al. [8]. The ten-second interval is the knee of a trade-off: it bounds the time to catch a persistent injection at one poll while the cost of dumping three small flow tables every ten seconds is negligible, and a shorter interval narrows but cannot close the window, since a rule injected and withdrawn inside one poll still evades it. Separately, every state-changing control operation, such as disarming the defence, requires an authentication token, so a host that reaches the control interface cannot silently switch CARS off; an unauthenticated attempt is refused and logged.

3.7 Deployment

CARS is deployed across the four machines of the testbed as shown in Figure 3.13. The arrangement keeps the two planes apart: the controller, which decides, is separate from the switches, which enforce, and the sensing sits with the fabric so that detection stays close to the traffic while the decision stays central.

The controller runs on Dell 2 as an os-ken application, `cars_engine.py`, launched with link observation enabled. It holds the whole decision logic described in Section 3.5, speaks OpenFlow 1.3 to the switches on port `6653`, and exposes the control interface used for responses and for the guarded operations on port `8080`. Nothing else of CARS runs on this node. It runs continuously and prints each decision as it is taken; a live extract is shown in Figure 3.15, and its status interface (Figure 3.14) reports the three switches connected and both guards enabled.

The fabric and the sensing run on Dell 1. The two Open vSwitch bridges, `ovs1` for Cell-1 and `ovsgw` for the gateway, carry the data plane. The Snort sensor reads a mirror of the gateway traffic and hands its findings to a small bridge, `cars-bridge`, which posts them to the controller at `/cars/respond`; this is the detection path of Section 3.6. The bridge de-duplicates, posting at most one event per source, conduit and operation every three seconds so a sustained attack cannot storm the controller with identical alerts, and it estimates each operation's arrival rate over that same three-second window for the volumetric overlay. Three supporting services run alongside: the flow-integrity checker, `cars-flowaudit`, which polls the tables every ten seconds against the trusted baseline; the remediation agent on `192.168.2.45`; and the decision dashboard. The operator, attacker and service endpoints are hosted in Linux network namespaces on this node, which is why a single machine can present the operator station, the Modbus device, the honeypot and the attacker vantage on distinct addresses.

Cell-2 runs on Dell 3 with its own switch, `ovs2`, and the address-translation clone that lets it re-use the Cell-1 plan, as described in Section 3.2. The Windows machine runs TIA Portal and Factory IO; it is both the engineering workstation, at `192.168.2.55`, and the physical process of the hardware-in-the-loop. One boundary follows from this layout: the Snort mirror is on the gateway, so traffic that stays within Cell-2 on `ovs2` is enforced by the proactive policy but is not seen by the detector. The reactive layer

therefore covers the paths that cross the gateway, which is where the supervisory and cross-cell conduits run.

Figure 3.3: The fabric read from the switches with `ovs-vsctl show`: Cell-1 (`ovs1`, top) and Cell-2 (`ovs2`, bottom). Each bridge is connected to the controller at `tcp:10.10.10.1:6653` with `fail_mode: secure`, and carries its PLC and HMI on separate ports; `ovs1` also holds the patch to the gateway and `ovs2` the internal `cell2gw`. Captured live on 3 August 2026 (OVS 3.3.4); the arrows are annotations.

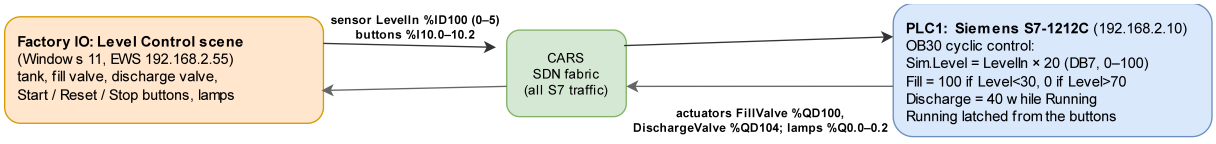

Figure 3.4: The hardware-in-the-loop. Factory IO supplies the level and button inputs and renders the valve and lamp outputs; PLC1 runs the control in OB30; every exchange crosses the CARS fabric. Addresses and mappings are from the deployed project.

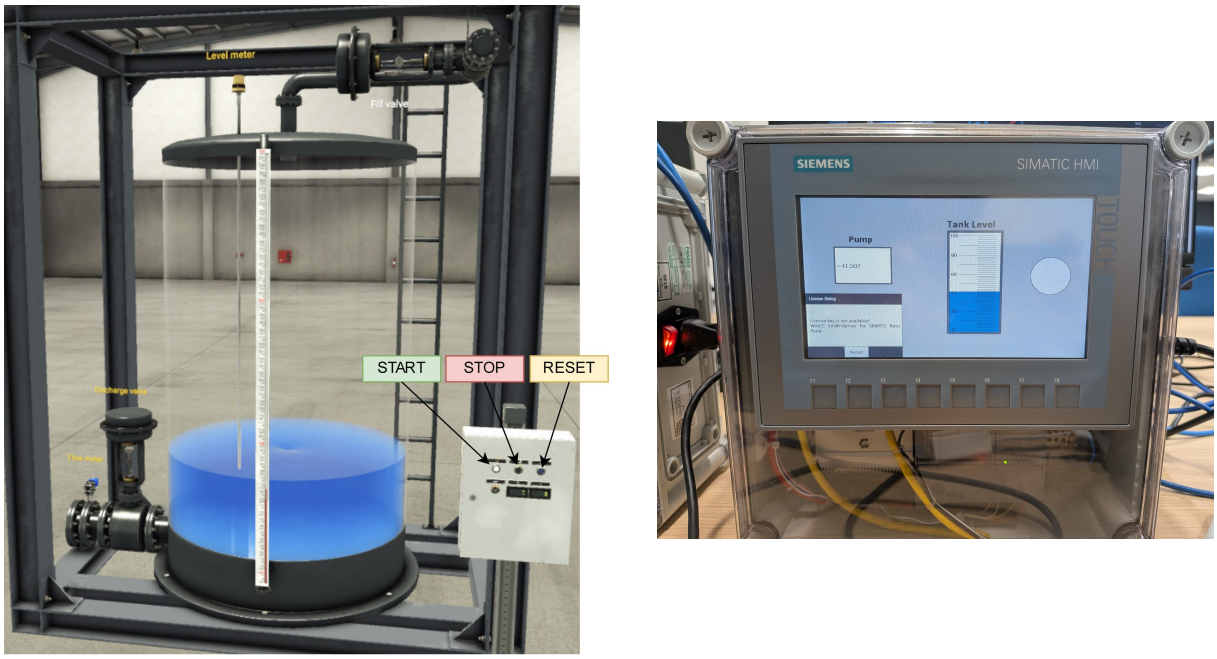

Figure 3.5: The live process. Left: the Factory IO tank scene, with the level meter, the fill and discharge valves, the flow meter, and the Start, Stop and Reset station. Right: the operator HMI reading the tank level, here about 40 per cent, and the pump value. The scene runs on the Windows machine and exchanges its inputs and outputs with PLC1 over S7 across the fabric.

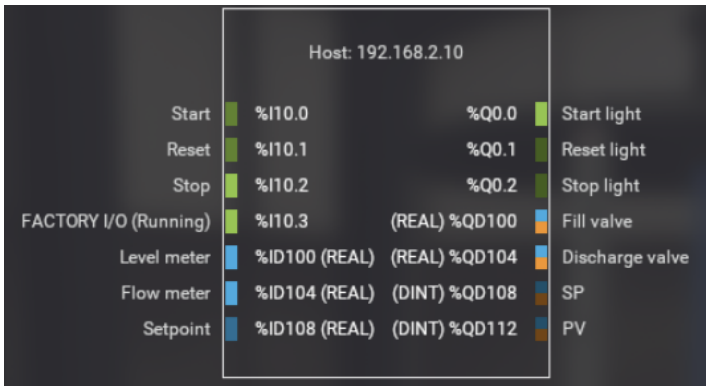

Figure 3.6: The Factory IO S7-1200 driver, bound to PLC1 at `192.168.2.10`. The inputs are the Start, Reset and Stop buttons at `%I10.0`–`%I10.2`, the running flag at `%I10.3`, and the level, flow and setpoint values at `%ID100`/`%ID104`/`%ID108`; the outputs are the lamps at `%Q0.0`–`%Q0.2`, the fill and discharge valves at `%QD100`/`%QD104`, and the setpoint and process value at `%QD108`/`%QD112`. Read from the deployed driver configuration; these match the OB30 tag table.

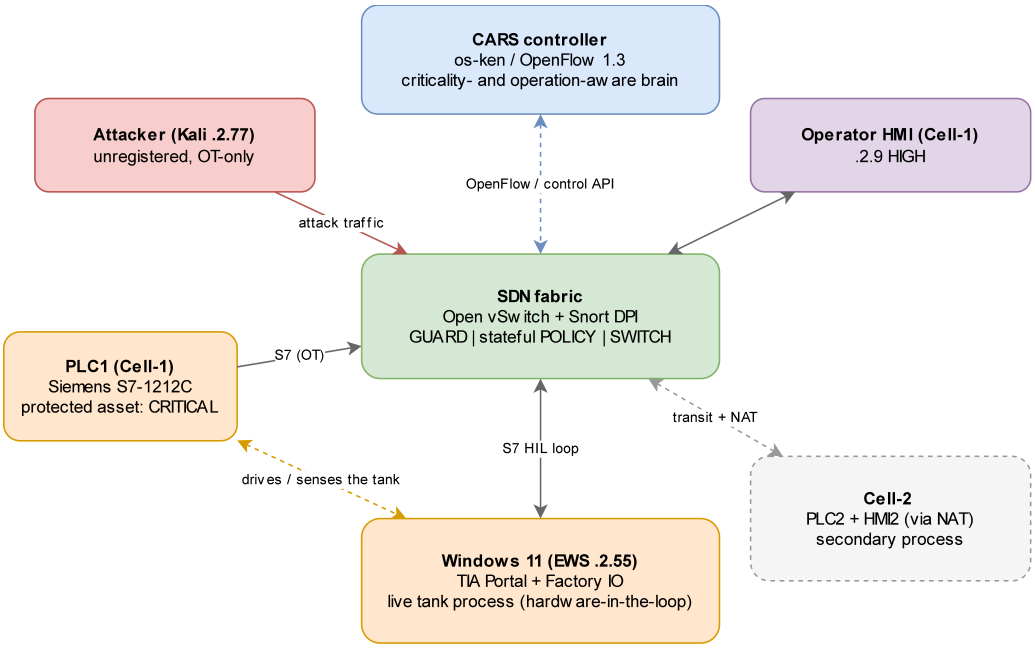

Figure 3.7: CARS in the fabric. All traffic to a protected asset crosses the SDN fabric, which the controller governs; the Factory IO process closes a hardware-in-the-loop with PLC1.

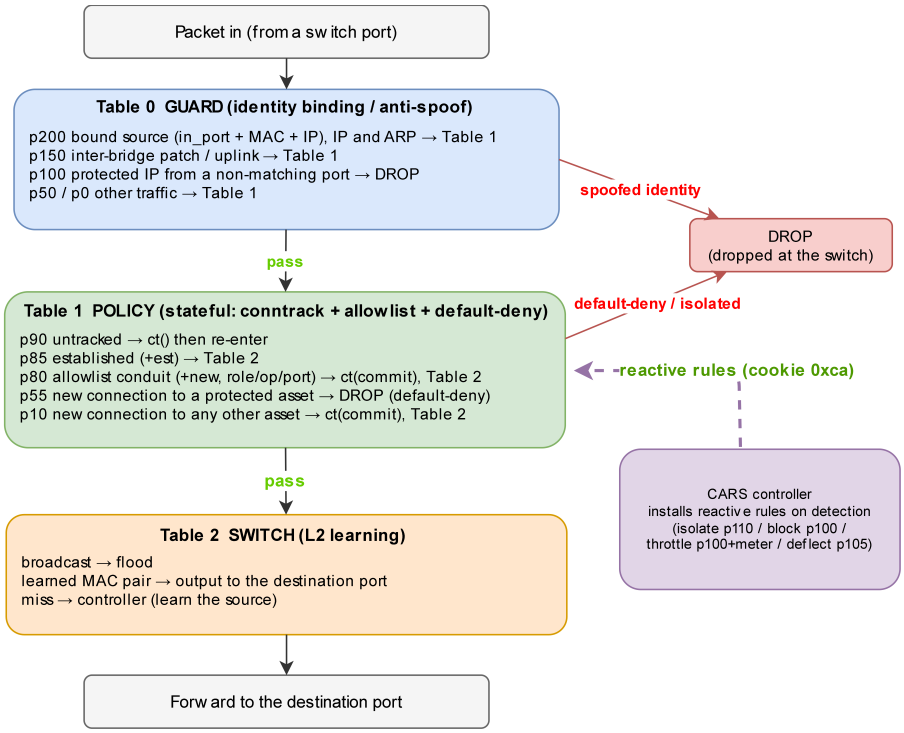

Figure 3.8: The CARS OpenFlow pipeline, captured live on `ovs1` and `ovsgw`. A packet is admitted only if its identity binds at Table 0, its connection is permitted and tracked at Table 1, and it is then forwarded at Table 2. Proactive policy carries cookie `0x00a2`; reactive responses installed by the controller carry `0x00ca`.

```
msclab@msc-lab:~/cars$ curl -s $API/cars/criticality
{"criticality": {"192.168.2.10": "CRITICAL", "192.168.3.10": "HIGH", "192.168.2.9": "HIGH", "192.168.3.9": "MEDIUM", "192.168.2.30": "MEDIUM", "192.168.2.20": "LOW"}, "weights": {"CRIT
ICAL": 3, "HIGH": 2, "MEDIUM": 1, "LOW": 0}}msclab@msc-lab:~/cars$
```

Figure 3.9: The criticality read live from the controller with `curl $API/cars/criticality`, backing Table 3.1: the six tiered assets and the weight of each tier. Captured on 3 August 2026.

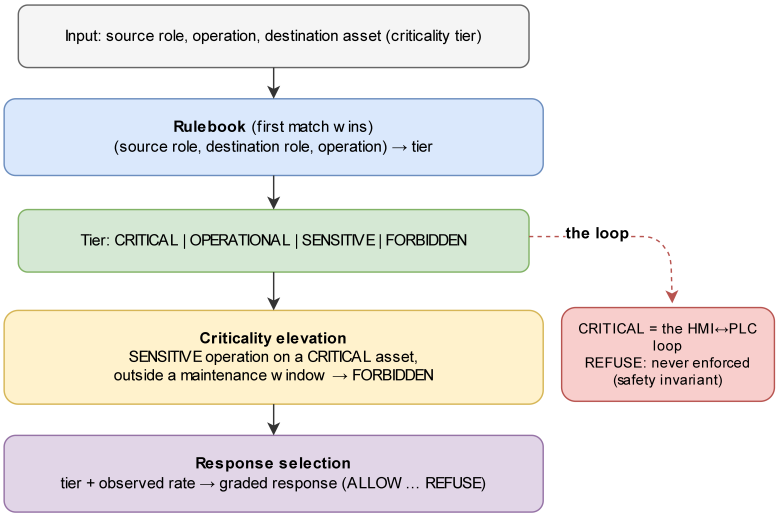

Figure 3.10: The CARS operation decision. The source role and operation give a tier by first match in the rulebook; a sensitive operation on a critical asset is elevated to forbidden; the tier and the observed rate then select the graded response.

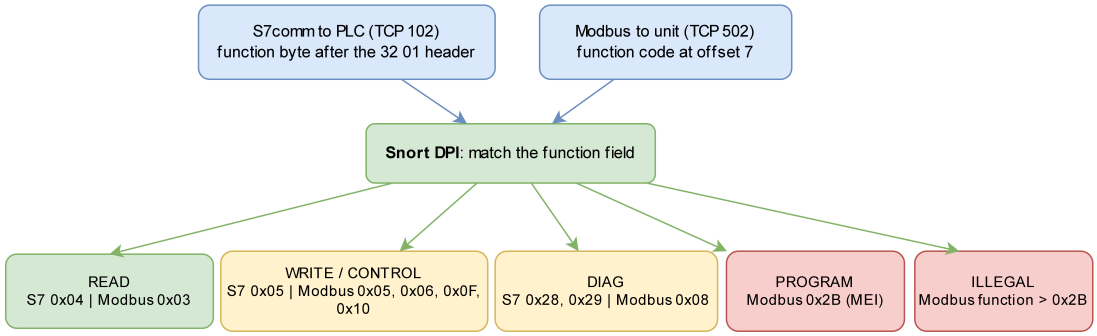

Figure 3.11: Operation classes recovered by deep packet inspection from the S7comm and Modbus function fields, as encoded in the deployed Snort rules.

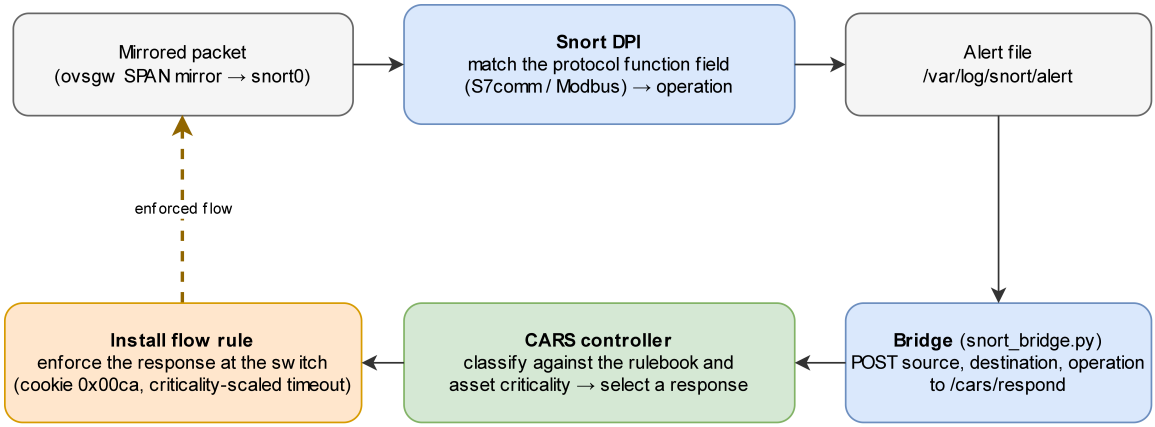

Figure 3.12: The detection-to-response path: a mirrored packet is classified by Snort, the alert is forwarded to the controller, which decides against the rulebook and criticality and installs the flow rule that enforces the response.

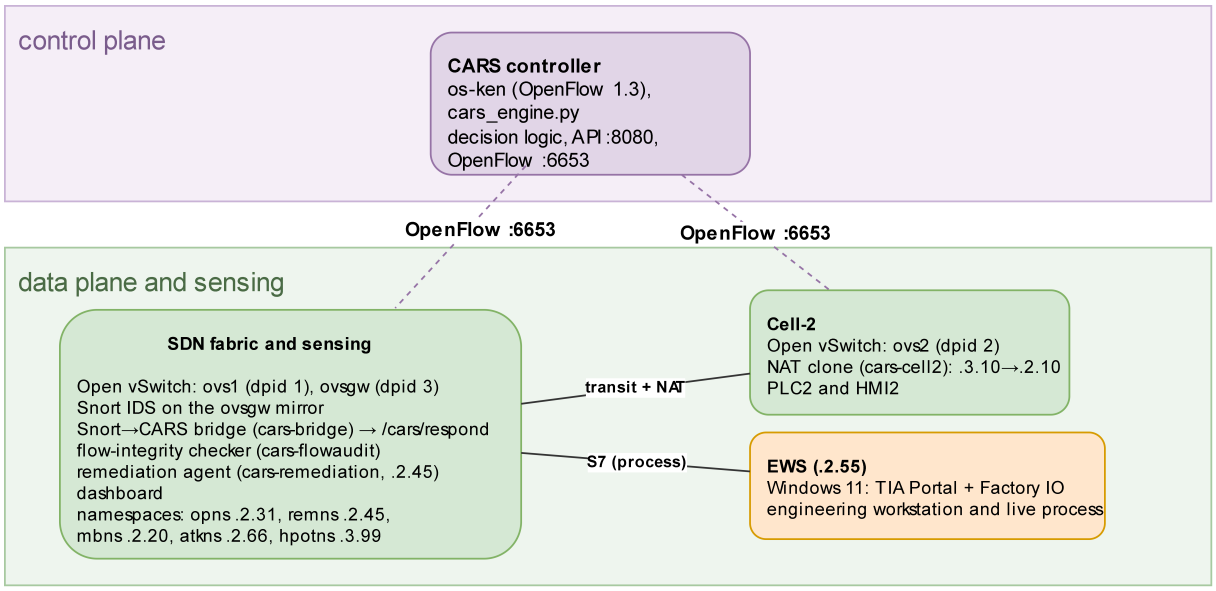

Figure 3.13: Deployment. The controller on Dell 2 governs the switches on Dell 1 and Dell 3 over the OpenFlow control plane; the fabric, the Snort sensor and the supporting agents run together on Dell 1, and the engineering workstation with Factory IO is the live process.

```
msclab@msc-lab:~/cars$ API=http://127.0.0.1:8080
msclab@msc-lab:~/cars$ curl -s $API/cars/status
{"switches": [2, 1, 3], "guard": true, "arp_guard": true, "cars_ms_avg": 0.026, "cars_ms_n": 35352, "mac_blocks": [], "conduit_blocks": []}
```

Figure 3.14: The controller status from `curl $API/cars/status`: the three switches connected (datapath identifiers 2, 1, 3), identity and ARP guards enabled, and the running mean decide time. Captured on 3 August 2026.

```
msclab@msc-lab: ~
BRAIN: 08-03T19:52:18  OPERATIONAL 192.168.2.55(ews) -> 192.168.2.10(plc) S7    READ =>  ALLOW (operational) - monitor only [CRIT:CRITICAL]
CARS decide+enforce: 0.033 ms
BRAIN: 08-03T19:52:20  OPERATIONAL 192.168.2.55(ews) -> 192.168.2.10(plc) S7    CONTROL =>  ALLOW (operational) - monitor only [CRIT:CRITICAL]
CARS decide+enforce: 0.021 ms
BRAIN: 08-03T19:52:21  OPERATIONAL 192.168.2.45(remediation) -> 192.168.2.10(plc) S7    READ =>  ALLOW (operational) - monitor only [CRIT:CRITICAL]
CARS decide+enforce: 0.026 ms
BRAIN: 08-03T19:52:21  OPERATIONAL 192.168.2.55(ews) -> 192.168.2.10(plc) S7    READ =>  ALLOW (operational) - monitor only [CRIT:CRITICAL]
CARS decide+enforce: 0.020 ms
BRAIN: 08-03T19:52:22  OPERATIONAL 192.168.3.66(historian) -> 192.168.3.10(plc) S7    READ =>  ALLOW (operational) - monitor only [CRIT:HIGH]
CARS decide+enforce: 0.022 ms
BRAIN: 08-03T19:52:22  OPERATIONAL 192.168.3.66(historian) -> 192.168.3.10(plc) TCP    =>  ALLOW (operational) - monitor only [CRIT:HIGH]
CARS decide+enforce: 0.043 ms
BRAIN: 08-03T19:52:22  OPERATIONAL 192.168.2.30(historian) -> 192.168.2.10(plc) S7    READ =>  ALLOW (operational) - monitor only [CRIT:CRITICAL]
CARS decide+enforce: 0.011 ms
BRAIN: 08-03T19:52:23  OPERATIONAL 192.168.2.55(ews) -> 192.168.2.10(plc) S7    CONTROL =>  ALLOW (operational) - monitor only [CRIT:CRITICAL]
CARS decide+enforce: 0.022 ms
BRAIN: 08-03T19:52:24  OPERATIONAL 192.168.2.55(ews) -> 192.168.2.10(plc) S7    READ =>  ALLOW (operational) - monitor only [CRIT:CRITICAL]
CARS decide+enforce: 0.037 ms
BRAIN: 08-03T19:52:24  OPERATIONAL 192.168.2.45(remediation) -> 192.168.2.10(plc) S7    READ =>  ALLOW (operational) - monitor only [CRIT:CRITICAL]
CARS decide+enforce: 0.039 ms
BRAIN: 08-03T19:52:26  OPERATIONAL 192.168.3.66(historian) -> 192.168.3.10(plc) S7    READ =>  ALLOW (operational) - monitor only [CRIT:HIGH]
CARS decide+enforce: 0.062 ms
BRAIN: 08-03T19:52:26  OPERATIONAL 192.168.3.66(historian) -> 192.168.3.10(plc) TCP    =>  ALLOW (operational) - monitor only [CRIT:HIGH]
```

Figure 3.15: The deployed controller running live on Dell 2. Each line records one decision: the source and destination, the protocol and operation, the verdict and response, the asset criticality, and the decide-and-enforce time. Captured from the controller console on 3 August 2026, confirming the engine runs as a service and logs every decision.

Chapter 4

Critical Evaluation

4.1 Method and metrics

The first question for a reactive defence in an operational network is not whether it can block an attack but whether it can be trusted not to block legitimate work. A false positive that cuts a control conduit is the deployment barrier, so the evaluation measures decision accuracy across the whole space of source role, operation and target criticality on the testbed, and reports the false-positive rate as the headline figure.

Each case is a tuple of source, destination, operation and rate, carrying a ground-truth label assigned from security intent: legitimate, attack, grey (criticality-graded) or flood-exempt. The measured decision is the deployed engine's own verdict, obtained by posting the case to the controller's `/cars/respond` interface, which runs the exact production path: the first-match classification of (role, operation) to a tier, the criticality elevation of a SENSITIVE operation on a CRITICAL asset, and the graded response selection. The engine is not reimplemented for the test; the labelled set is driven through it. The pass runs with enforcement disarmed, so it measures pure decisions with no side effect on the live conduits, then re-arms.

The verdicts follow the intent. A legitimate case is a true negative if it is permitted, that is ALLOW, MONITOR, or REFUSE, the last being the safety loop that is mirrored and alerted but never cut, and a false positive if it is restricted. An attack case is a true positive if it is restricted, that is THROTTLE, DEFLECT, ISOLATE or BLOCK, and a false negative if it is permitted. Grey cases are shown to demonstrate grading rather than scored: they test how the response scales with criticality, not a binary legitimate-or-attack decision, so they carry no true or false label by construction. All three are nonetheless handled correctly, and including them would leave the accuracy unchanged. The matrix was regenerated on the live system for this writeup and reproduces the stored baseline row for row; the decisions are corroborated at the packet level by the campaign of Section 4.4, so the verdicts are not decision-only artefacts.

Several attacker identities recur across the evaluation and are named consistently. The unregistered on-segment attacker occupies the namespaces `192.168.2.66` and `192.168.2.67` and drives most of the campaign; the compromised but allowlisted supervisory host `192.168.2.31` launches the insider false-data injection; and a dual-homed Kali virtual machine provides the two-pivot trace of Section 4.7, seen on the OT segment as `192.168.2.77` and, when routed through the modelled perimeter, collapsed by network-address translation onto the single gateway identity `192.168.2.1`. Each figure is captioned with the identity of the run it shows.

4.2 Decision accuracy and false positives

Table 4.1 gives the full case set as measured. Of the twenty-four scored cases every one is classified correctly: thirteen true positives, eleven true negatives, no false positives and no false negatives, a decision accuracy of 100% and, the figure that matters for deployment, a false-positive rate of 0%. The confusion summary is Table 4.2. The full case set, with the inputs and the verdict for each case, is reproduced in Appendix A.5. The twenty-four is not an arbitrary sample but the closure of the decision space: one case per reachable branch of the classification, elevation and response-selection path, so that every rule the engine can apply is exercised at least once and none is duplicated. A twenty-fifth case on the same branches would add no coverage. This is exhaustive coverage of the policy as deployed on this testbed,

Src→Dst	Operation	Tier	Response	V	Case
Legitimate benign (false-positive test)					
9→10	READ	CRITICAL	REFUSE	TN	HMI→PLC1 read (loop)
9→10	WRITE	CRITICAL	REFUSE	TN	HMI→PLC1 setpoint (loop)
10→9	READ	CRITICAL	REFUSE	TN	PLC1→HMI reply (loop)
55→10	READ	OPERATIONAL	ALLOW	TN	EWS/Factory IO read (HIL)
55→10	CONTROL	OPERATIONAL	ALLOW	TN	EWS/Factory IO write (HIL)
45→10	CONTROL	OPERATIONAL	ALLOW	TN	Remediation restore
30→10	READ	OPERATIONAL	ALLOW	TN	Historian poll PLC1
30→20	READ	OPERATIONAL	ALLOW	TN	Historian poll Modbus
31→20	READ	OPERATIONAL	ALLOW	TN	SCADA Modbus poll
31→10	READ	OPERATIONAL	ALLOW	TN	SCADA read PLC1
Grey, criticality-graded (shown, not scored)					
31→10	WRITE	FORBIDDEN	ISOLATE	grey	SCADA write CRITICAL (elevated)
31→20	WRITE	SENSITIVE	THROTTLE	grey	SCADA write LOW Modbus
30→9	WRITE	SENSITIVE	THROTTLE	grey	Historian write HIGH HMI1
Attacks (true-positive / false-negative test)					
77→10	TCP	FORBIDDEN	ISOLATE	TP	Kali outsider TCP
77→10	READ	FORBIDDEN	ISOLATE	TP	Kali outsider read
66→20	WRITE	FORBIDDEN	BLOCK	TP	Unknown write Modbus
31→10	CONTROL	FORBIDDEN	ISOLATE	TP	Compromised SCADA actuates
31→10	PROGRAM	FORBIDDEN	ISOLATE	TP	SCADA program download
55→10	PROGRAM	FORBIDDEN	ISOLATE	TP	EWS program download (no window)
55→10	DIAG	FORBIDDEN	ISOLATE	TP	EWS diagnostics
55→10	ILLEGAL	FORBIDDEN	ISOLATE	TP	Malformed S7
1→10	READ	FORBIDDEN	ISOLATE	TP	Gateway reaches PLC (no conduit)
30→10	CONTROL	FORBIDDEN	ISOLATE	TP	Historian issues CONTROL
66→10	READ	FORBIDDEN	ISOLATE	TP	Unknown read PLC1
Flood (volumetric overlay)					
30→10	READ (50)	OPERATIONAL	BLOCK	TP	Historian read flood
55→10	CONTROL (50)	OPERATIONAL	ALLOW	TN	EWS high-rate (flood-exempt)
31→20	READ (50)	OPERATIONAL	THROTTLE	TP	SCADA read flood on Modbus

Table 4.1: The decision matrix, measured live from the deployed engine via `/cars/respond`. Hosts are `192.168.2.x` (last octet shown). Twenty-four scored cases: 13 TP, 11 TN, 0 FP, 0 FN; three grey cases show grading. Regenerated for this writeup; identical to the 3 August baseline. Full per-decision controller log in Appendix A.6.

	Permitted	Restricted
Actually legitimate (11)	11 (TN)	0 (FP)
Actually attack (13)	0 (FN)	13 (TP)

Table 4.2: Confusion summary. Accuracy 24/24 = 100%; false-positive rate 0/11 = 0%; false-negative rate 0/13 = 0%.

not a claim that the same rulebook generalises unchanged to another plant's assets and operations: it establishes correctness of the decision logic here, not deployment-grade confidence for industrial traffic at large.

Coverage establishes correctness on every branch; a larger run establishes that it holds at scale. A corpus of 2,078 labelled cases was driven through the same decision-only path with the harness `vast_accuracy.py`: fifty-one unregistered attacker sources against an asset of every tier, across read, write, control and diagnostic operations at normal and flood rates, giving 2,040 attack cases, together with the legitimate supervisory conduits at sub-flood rates and the flood-exempt high-rate input, which form the false-positive test. The engine classified all 2,078 correctly: 2,040 true positives, 38 true negatives, no false positives and no false negatives, a decision accuracy of 100% with a 95% Wilson confidence interval of $[0.9982, 1.0000]$. Not one of the 2,040 attack variants was permitted, and no legitimate or exempt case was cut. The false-negative result carries statistical weight on its own count of 2,040; the zero false-positive rate rests on these 38 legitimate and exempt cases together with the live campaign of Section 4.4, because the legitimate input space is bounded by the allowlist rather than by the number of trials. Like the scored matrix, this remains a single-platform, single-policy validation on the same testbed: it strengthens confidence in the decision logic itself, but it is not evidence that the result transfers to a different fabric, asset mix or rulebook, and it should not be read as production-proven.

Every legitimate operation is permitted, including the two that most tempt a blunt filter. The high-rate control stream from the engineering and Factory IO host is flood-exempt and passes at fifty writes per second, because that host drives the hardware-in-the-loop and its rate is expected. The operator loop

between HMI and PLC is tier CRITICAL and carries the REFUSE marker, so it is watched but never enforced against. The 0% false-positive rate on this traffic is the operator-trust result: the defence can be armed on the live process without cutting the work that keeps it running.

The three grey cases show the criticality grading on one operation class. A SCADA write to the CRITICAL PLC1 is elevated from SENSITIVE to FORBIDDEN and quarantined; the same role's write to the LOW Modbus unit stays SENSITIVE and is throttled; a historian write to the HIGH operator panel is throttled. The response scales with the target's criticality, not the operation alone.

The set also isolates the operation-awareness a five-tuple filter cannot reach. The identical conduit from SCADA to the Modbus unit draws three different verdicts by operation and rate alone: a read is allowed, a write is throttled as a SENSITIVE actuation, and a read driven to fifty per second is throttled as a volumetric abuse. The decision is made on the industrial operation, not the addresses and ports.

The matrix is exhaustive over the meaningful decision space of the testbed rather than a random sample: it covers every role, operation and criticality combination that arises here. The same verdicts are then corroborated on live traffic by the campaign of Section 4.4, where the deployed engine judged every phase and logged on the order of a thousand decisions per run, and the legitimate loop and telemetry were permitted throughout with no false positive observed (Appendix A.6). The 100% figure is therefore a statement about this system's decision logic, established over the decision space and confirmed on live traffic, not a statistical guarantee for an arbitrary production network; that scope is stated in Section 4.11.

To probe further, the engine was driven across its entire decision domain, every source, asset, operation and rate combination on the testbed, 868 cases in all. It returned a defined verdict for each, and on inspection every verdict is consistent with the deployed policy: the unregistered attacker is isolated on every operation against every asset, high-rate operations are restricted as floods, non-allowlisted conduits are default-denied, and the supervisory roles keep their reads. Building an independent oracle for a policy this nuanced is itself error-prone, so this exhaustive pass is a totality and consistency check rather than a second accuracy figure. The one behaviour it highlights is that the trusted remediation agent may issue even dangerous operations to the PLC unblocked, the trusted-role exposure of Section 4.11.

4.3 Criticality-graded response

The criticality model earns its place only if the target's importance changes the outcome. The sweep, rerun with `cars_criticality_proof.sh`, holds the actor and the operation fixed and varies only the target's tier, in three parts: the bounded elevation of the decision, the proportionality of the response, and the invariants that criticality must never break.

Elevation is bounded. A SENSITIVE operation on a CRITICAL asset is raised to FORBIDDEN, as the grey cases of Table 4.1 show for a SCADA write to PLC1, whereas a read is never elevated: the same SCADA reading the same CRITICAL PLC1 stays OPERATIONAL and is allowed. The elevation is specific to the critical tier: the same actuating write against the HIGH PLC2, weight two, is not raised to FORBIDDEN but stays SENSITIVE and is rate-limited, so the two assets are handled differently by design, which the four-tier sweep and the two-hundred-and-forty-decision run below exercise directly. The model suspends this elevation within an authorised maintenance window (Section 3.5), so scheduled engineering is not blocked by the rule that protects the asset at other times.

Proportionality is measured on the response side. The same forbidden control, from the same harmless attacker, draws a reactive rule whose length scales with the target's weight: 75 seconds on the CRITICAL PLC1, 60 seconds on the HIGH PLC2, 45 seconds on the MEDIUM historian and 30 seconds on the LOW Modbus unit, each self-healing when it expires (Figure 4.1). The duration follows $30 + 15w$ seconds for tier weight w, measured at all four tiers. The scaled quantity is the timeout; it is applied to whichever reactive rule the escalation state selects, a source ISOLATE once an attacker persists, or a single-conduit BLOCK on a first offence, as the MEDIUM case shows.

A larger run confirms the grading holds across operations and in combination, not one case at a time. Two hundred and forty decisions were driven through the deployed engine: an unregistered attacker and a compromised but trusted supervisory host, against all four tiers, across read, write, control and diagnostic operations, at normal and flood rates, and in both alternating and simultaneous bursts. The timeout tracks the target's tier throughout, 75, 60, 45 and 30 seconds, and the engine returned a correct verdict for every tier even when all four were attacked at once. The source is judged by what it does rather than by a blanket rule: the unregistered attacker is isolated on every operation, whereas the trusted host keeps its reads and is cut only on the operations its role must not perform. A flood withdraws the leniency a normal rate earns, so a permitted read on the critical asset is escalated from allowed to cut, and a source

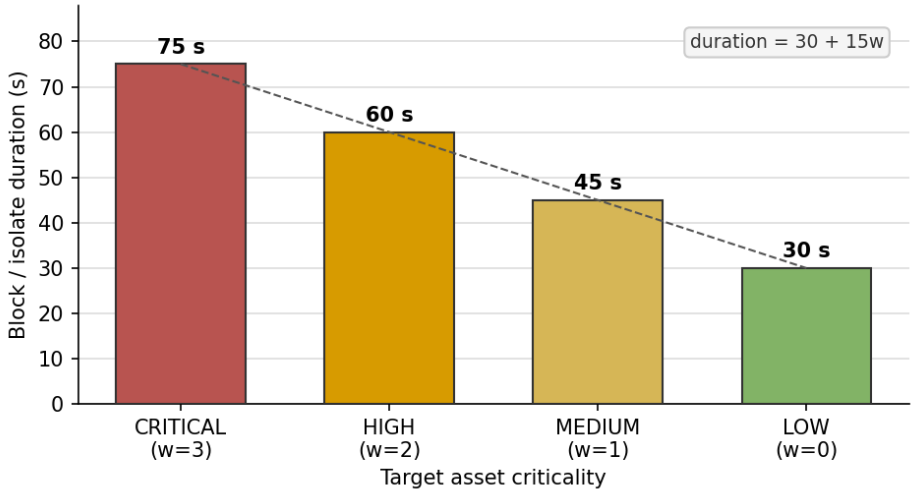

Figure 4.1: The reactive rule duration measured live for an identical forbidden control against targets of each criticality tier: 75, 60, 45 and 30 seconds for weights 3, 2, 1 and 0, following $30 + 15w$. Same actor and operation throughout; only the target's tier changes.

that keeps offending is treated more harshly as its offence count rises. The grading is stable across the full range of operation, rate, source and concurrency, not only the single-operation sweep of Figure 4.1.

The invariants hold throughout. The operator control loop is never enforced against, even on the CRITICAL PLC1, where an HMI-to-PLC control returns REFUSE and is only mirrored and alerted, the safety cap of the model. Trusted monitoring is always allowed, even on the CRITICAL asset, where a SCADA read returns OPERATIONAL and passes. Criticality raises the response to a threat but never overrides the two rules that keep the process controllable.

4.4 Wire-level campaign: armed versus unprotected

The decision accuracy of Section 4.2 is necessary but not sufficient; a decision matters only when it is enforced on the wire against a real attacker. This campaign exercises the defence at the packet level, comparing the armed system against the unprotected process. Unprotected has two honest meanings, kept separate: *disarmed*, where reactive enforcement is switched off but CARS still detects and decides and the proactive structure stands, used for the insider and process vectors; and a true *bypass* around the governed pipeline, used for the outsider vectors, since the proactive default-deny is always in force and merely disarming does not expose an unregistered host. Each vector is captured at three wire points, the PLC S7 port, the OpenFlow control channel and the Snort mirror. The legitimate Factory IO loop and the historian telemetry ran throughout and were permitted in every state, so the campaign carried zero false positives from end to end. Figure 4.2 shows what this means for a single attack packet in each state.

4.4.1 Process devastation and the deception

The centrepiece is a false-data injection on the live tank, the attack that most directly threatens a physical process. A compromised supervisory host, `192.168.2.31`, writes the level sensor and the discharge valve in the process image over S7 at high rate, driving the tank past its safe band. Each write is a CONTROL operation to the CRITICAL PLC1. The disarmed and armed cases are compared on the same attacker, the same operation and the same target.

Disarmed, the injection runs unimpeded and the process is devastated. The tank is driven clean over the top: the Factory IO vessel fills completely and the operator panel's high-level alarm fires, the level cresting past the full mark at about 104% (Figure 4.3). A sensor-spoof variant of the same attack instead pins the reported level near zero, so the operator sees a safe, empty tank while the real one fills, the deception at the heart of a process attack (Appendix A.8). Either way, while the attacker holds the actuators the operator cannot regain control.

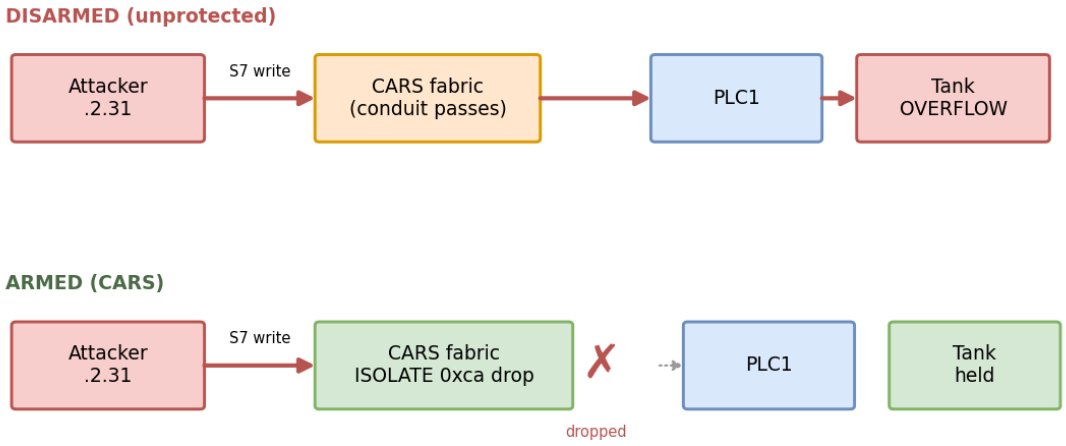

Figure 4.2: The path of a malicious S7 write in each state. Disarmed, the write crosses the fabric, reaches PLC1 and drives the tank to overflow. Armed, the reactive rule (`cookie 0x00ca`) drops it in the fabric, so it never reaches the PLC and the process is held.

Armed, the same attack is severed before it can move the process. The injection's writes are CONTROL operations to the CRITICAL PLC1; CARS classifies them FORBIDDEN and isolates the source. The attacker's client cannot land a write that moves the process. In this run its S7 session is cut before any write commits, none landing against the 973 disarmed, and a second attempt cannot even open a connection, its traffic already dropped. The wire-level proof below (Figure 4.7) is exact about the boundary on a conduit that is already open: at most a single first-packet frame reaches the PLC port before the isolate installs, and the tank absorbs it without leaving its band, so no write reaches the process in a state that can move it. The enforcement is a real installed rule, not a log entry. On every switch the controller installs `cookie=0xca`, `table=1`, `priority=110`, `nw_src=192.168.2.31`, `actions=drop` with a 75-second self-healing timeout, the CRITICAL duration (Figure 4.5); the controller's own decision log shows the same event across the layer as a `FORBIDDEN ⇒ ISOLATE (ENFORCED)` verdict on `192.168.2.31` (Figure 4.4), and the rule self-expires after its 75-second window. With no write reaching the PLC the tank never leaves its band and the process stays under the PLC's control. This is block-and-maintain: the attack is severed at the datapath before it can move the process. The dashboard itself, how it discovers the topology from the controller and renders each live decision, is described in Appendix A.10.

The contrast is stark on both the network and the process (Figure 4.6). The identical FDI injection, sourced from the compromised supervisory host `192.168.2.31` and run over a matched forty-second window, landed 973 writes at the PLC disarmed against none armed; disarmed the tank crested past the full mark and overflowed, the process consequence of an unblocked injection shown in Figure 4.3, and armed it stayed within its safe band. The armed zero is not a near miss but a clean severance: the source is classified FORBIDDEN and isolated before its S7 session commits a single write.

4.4.2 The rest of the repertoire

The remaining vectors ran in the same armed session, captured at the three wire points. Against the control plane, an unauthenticated request to disarm the defence, `POST /cars/defense {on:false}`, was refused with `HTTP 401` and logged as `DENIED (bad/missing token)`; CARS stayed armed, so a compromised control-plane host cannot silently switch the defence off. A flow tamper that injected a bogus rule and deleted a real conduit was caught by the flow-integrity checker within one ten-second poll, its verdict moving to `ok:0, missing:1, extra:1`, and cleared back to `ok:1` on restore without re-baselining, which shows the restoration was byte-exact. An out-of-state attack, forged ACKs from `192.168.2.66` to the PLC, did not reach it: the stateful table admitted none. And the operation-aware case, a compromised SCADA writing a bogus level to the PLC, was isolated on the write while the remediation agent restored the value, the level holding near 50 and the loop still running, the same block-and-maintain as the devastation phase. Through all of it the legitimate Factory IO loop and the

Figure 4.3: Process devastation, disarmed. Left: the Factory IO tank driven full and over the top. Right: the operator panel at the same moment, the level cresting past the full mark at +104.326 with the high-level alarm raised. The attacker holds the process out of its safe band and the operator cannot recover it.

Figure 4.4: The same armed isolation seen at the decision layer on the CARS dashboard: the compromised supervisory host 192.168.2.31 (scada) issuing S7 CONTROL to the CRITICAL PLC1 is classed FORBIDDEN and met with an ENFORCED ISOLATE. This is the cross-layer counterpart to the datapath rule of Figure 4.5: the same event as a policy decision rather than an installed flow.

historian telemetry were permitted, so the campaign carried no false positives from end to end.

The two external vectors close the campaign, driven from the Kali attacker 192.168.2.77, the unregistered outsider of the threat model. Reconnaissance learned nothing under CARS: every scanned port on the PLC, including the S7 port 102, returned filtered, the s7-info probe drew no model or firmware, and the scan was reactively isolated. The controller logged the scan as ISOLATE source 192.168.2.77 75s ...ENFORCED [CRIT:CRITICAL,] and the drop rule appeared on the switches as cookie=0xca, priority=110, nw_src=192.168.2.77. Identity spoofing was dropped at the door: forging ARP as the trusted seam 192.168.2.31, every forged frame was dropped at GUARD ingress and logged as REFUSED (identity-spoof) - 192.168.2.31 impersonation dropped at GUARD ingress (dpid3), while the real seam kept flowing (Appendix A.8). The whole campaign is summarised in Table 4.3.

4.5 Cross-layer validation

A decision log can be doubted; independent agreement across every layer is harder to dismiss. A deep end-to-end trial drove the full response spectrum through the real autonomous chain and recorded, for each scenario, five independent layers: the packet on the Snort mirror, the Snort alert, the controller's decision, the installed OpenFlow rule, and the outcome at the client. Table 4.4 gives the result; at every rung the layers tell the same story.

frame amongst the legitimate Factory IO loop traffic on the PLC port, the attack is a needle in the live process stream.

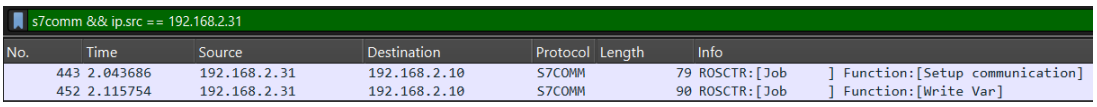

Figure A.4: Disarmed, the attacker's S7 Write Var (frame 452) reaches the PLC. With CARS armed the same write is dropped in the fabric (Figure 4.7).

436	2.013042	192.168.2.55	192.168.2.10	S7COMM	85	ROSCTR:[Job] Function:[Read Var]
437	2.026484	192.168.2.10	192.168.2.55	S7COMM	95	ROSCTR:[Ack_Data] Function:[Read Var]
441	2.044185	192.168.2.55	192.168.2.10	S7COMM	90	ROSCTR:[Job] Function:[Write Var]
442	2.055486	192.168.2.10	192.168.2.55	S7COMM	76	ROSCTR:[Ack_Data] Function:[Write Var]
444	2.056223	192.168.2.55	192.168.2.10	S7COMM	85	ROSCTR:[Job] Function:[Read Var]
449	2.071397	192.168.2.10	192.168.2.55	S7COMM	80	ROSCTR:[Ack_Data] Function:[Read Var]
451	2.072077	192.168.2.55	192.168.2.10	S7COMM	101	ROSCTR:[Job] Function:[Write Var]
454	2.079629	192.168.2.31	192.168.2.10	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
455	2.132453	192.168.2.10	192.168.2.55	S7COMM	76	ROSCTR:[Ack_Data] Function:[Write Var]
457	2.133267	192.168.2.55	192.168.2.10	S7COMM	85	ROSCTR:[Job] Function:[Read Var]
458	2.135428	192.168.2.10	192.168.2.31	S7COMM	81	ROSCTR:[Ack_Data] Function:[Setup communication]
459	2.135670	192.168.2.31	192.168.2.10	S7COMM	85	ROSCTR:[Job] Function:[Read Var]
462	2.156451	192.168.2.10	192.168.2.31	S7COMM	80	ROSCTR:[Ack_Data] Function:[Read Var]
465	2.161403	192.168.2.10	192.168.2.55	S7COMM	95	ROSCTR:[Ack_Data] Function:[Read Var]
469	2.178200	192.168.2.55	192.168.2.10	S7COMM	90	ROSCTR:[Job] Function:[Write Var]
470	2.194432	192.168.2.10	192.168.2.55	S7COMM	76	ROSCTR:[Ack_Data] Function:[Write Var]
472	2.195052	192.168.2.55	192.168.2.10	S7COMM	85	ROSCTR:[Job] Function:[Read Var]
473	2.207409	192.168.2.10	192.168.2.55	S7COMM	80	ROSCTR:[Ack_Data] Function:[Read Var]

Figure A.5: The attacker's Setup frame (`.31`→`.10`, highlighted) amongst the legitimate `.55` Factory IO loop traffic on the PLC port. CARS must pick the malicious operation out of a continuous stream of legitimate S7 reads and writes.

The enforcement itself is visible on a third wire, the OpenFlow control channel (TCP 6653), captured in `of_control.pcap`. Listing A.10 is the `OFPT_FLOW_MOD` that installs the reactive isolate, decoded from that capture: an ADD of the drop rule stamped with the CARS reactive cookie `0x00ca`, at priority 110 in the POLICY table, matching the attacker's source with a 75-second self-healing timeout. Two identical messages are sent, one to `ovs1` and one to `ovsgw`, so the quarantine is installed fabric-wide from a single decision. This is the control-plane counterpart to the datapath dump of Figure 4.5: the rule seen being installed, not just seen in place.

```
OFPT_FLOW_MOD              (version 0x04, type 14, control channel tcp/6653)
  command      = ADD
  cookie       = 0x00000000000000ca     (CARS reactive marker; 0xa2 = static A2 policy)
  table_id     = 1                        (POLICY)
  priority     = 110                       (above the allowlist and default-deny)
  hard_timeout = 75 s                      (CRITICAL asset: 30 + 15*3, then self-heals)
  match        = eth_type=0x0800, ipv4_src=192.168.2.31
  instructions = APPLY_ACTIONS []          (no output action = drop)
-> sent to ovs1 and ovsgw from one controller decision (fabric-wide quarantine)
```

Listing A.10: The reactive isolate as an `OFPT_FLOW_MOD`, decoded from `of_control.pcap` (OpenFlow 1.3, tcp/6653).

A.8 Scenario testing in full

The process devastation of Section 4.4.1 has a second, quieter variant: instead of driving the tank visibly over the top, the attacker pins the reported level near zero, so the operator panel shows a safe, empty tank while the real one fills. Figure A.6 captures this deception from a disarmed run: the real Factory IO tank stands filled while the HMI reads a level of zero and a pump of `+0.000`.

The external vectors were driven from the Kali attacker `192.168.2.77`. Reconnaissance under CARS is shown in Figure A.7: an armed port scan of the PLC returns every port `filtered`, and the scan is reactively isolated with a criticality-scaled quarantine. Identity spoofing is shown in Figure A.8: the GUARD anti-spoof drop counters register the forged ARP frames dropped at ingress. The reaction window over one hundred autonomous trials is in Figure A.9.

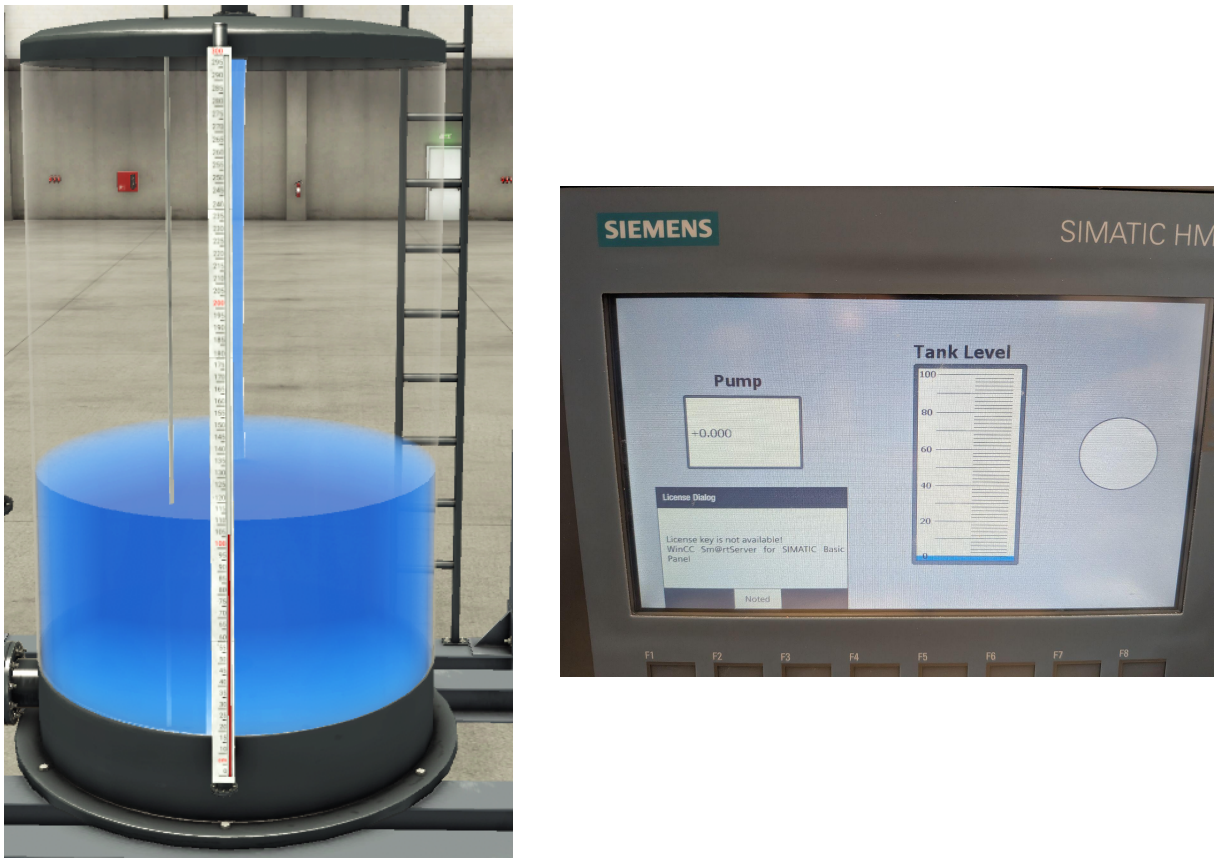

Figure A.6: The sensor-spoof deception, disarmed. Left: the real Factory IO tank, filled. Right: the operator HMI at the same moment, reading zero. The reported process has been divorced from the real one.

A.9 Scripts and code

Each script is presented as its role plus its load-bearing snippet. The lead is the controller, `cars_engine.py`: the whole evaluation drives its decision path, whose core is `classify()`, mapping a (source, destination, operation) to a tier by walking the rulebook first-match-wins.

```
def classify(src_ip, dst_ip, op=None):
    s, d = role_of(src_ip), role_of(dst_ip)      # map IP -> role from the REGISTRY
    for rsrc, rdst, rop, tier in RULEBOOK:        # rules in priority order
        if (rsrc == "any" or rsrc == s or rsrc == src_ip) and \
           (rdst == "any" or rdst == d or rdst == dst_ip) and \
           (rop  == "any" or rop  == op):
            return tier, s, d                     # first match wins -> its tier
    return "FORBIDDEN", s, d                      # nothing matched -> default deny
```

Listing A.11: The controller decision path (`cars_engine.py`): first-match rulebook classification.

The role lookup resolves each address to its registered role; the loop returns the tier of the first rulebook row whose source, destination and operation all match, and defaults to FORBIDDEN when none do. This is the exact path the accuracy evaluation drives through `/cars/respond`. The remaining load-bearing pieces of the controller and of the supporting scripts follow in the same form.

Controller: the rest of the decision-to-enforcement path

Criticality elevation and block duration (`respond()`). After classification, the destination's criticality weight drives two things: a trusted actuating write (SENSITIVE) to a CRITICAL asset is elevated to FORBIDDEN unless an authorised maintenance window is open, and the quarantine duration is scaled by the same weight.

```
acl = crit_of(dst_ip); dcw = CW.get(acl, 0)          # dst asset weight: 3..0
in_window = time.time() < self.maint_until
elevated = (tier == "SENSITIVE" and dcw >= 3 and not in_window)
```

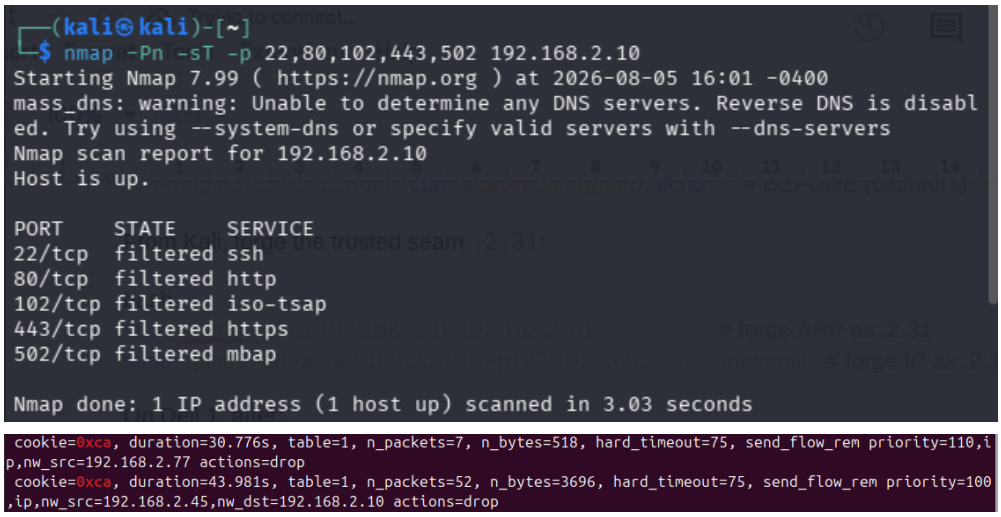

Figure A.7: Reconnaissance from Kali, armed. Top: the port scan of PLC1, every port `filtered`. Bottom: the reactive rule installed against the scanner, `cookie=0xca`, `priority=110`, `nw_src=192.168.2.77`, `actions=drop`, `hard_timeout=75`.

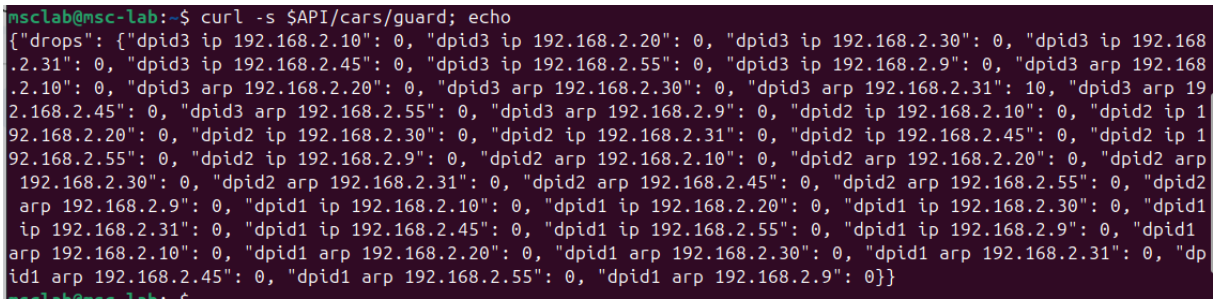

Figure A.8: Identity spoofing at GUARD. After forging ARP as the trusted seam `192.168.2.31` from Kali, the anti-spoof counter `dpid3 arp 192.168.2.31` registers the dropped frames (running total 10), every forged frame dropped at ingress with all other bindings untouched. The controller logged each as `REFUSED (identity-spoof)`.

```
if elevated:
    tier = "FORBIDDEN"          # trusted WRITE to a CRITICAL asset -> forbidden
timeout = BLOCK_TIMEOUT + dcw * 15   # CRITICAL 75, HIGH 60, MED 45, LOW 30 s
```

Listing A.12: Grey-zone elevation and criticality-scaled block duration.

The graded response ladder (`select_response()`). A tier, the asset weight and a rate flag select one of the seven responses; the safety loop is never enforced against, and higher-criticality assets escalate sooner and skip rungs.

```
if tier == "CRITICAL":    return "REFUSE"        # safety invariant - never enforce
esc = max(1, ESCALATE - dcw); c = state.get("count", 1)
if tier == "OPERATIONAL":
    if flood:
        if dcw >= 3: return "BLOCK"              # flood on CRITICAL asset -> cut
        return "THROTTLE" if c < esc else "BLOCK"
    return "ALLOW"                               # trusted conduit, normal rate
if tier == "SENSITIVE":
    if flood:    return "BLOCK"
    if dcw >= 3: return "BLOCK"                  # CRITICAL asset: skip THROTTLE
    return "THROTTLE" if c < esc else "BLOCK"
if flood:    return "ISOLATE"                    # FORBIDDEN
if dcw >= 3: return "ISOLATE"                    # CRITICAL asset: isolate now
return "BLOCK" if src_count < esc else "ISOLATE"
```

Listing A.13: Criticality-graded, flood-aware response selection.

The reactive install (`isolate_source()`). Every enforcement is a real OpenFlow rule stamped with the distinct reactive cookie `0x00CA`, self-expiring via `hard_timeout` and notifying the controller on removal

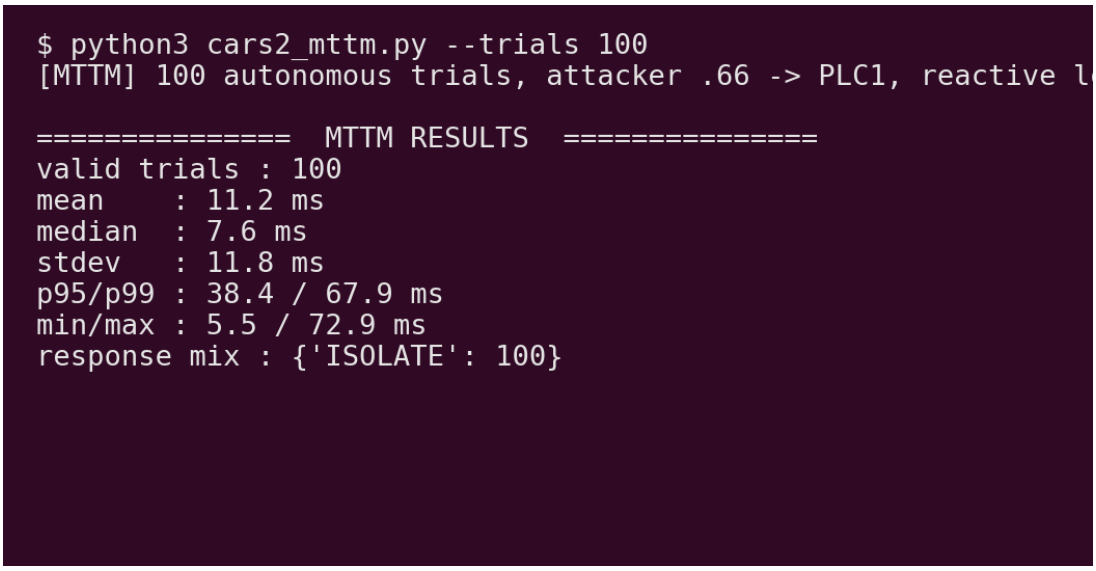

Figure A.9: Reaction window (mean-time-to-mitigate) over one hundred autonomous trials, attack-frame to enforcement on a single clock: median 7.6 ms, mean 11.2 ms, standard deviation 11.8 ms, minimum 5.5 ms, maximum 72.9 ms, every trial ending in ISOLATE.

(SEND_FLOW_REM) so the defence self-heals when the attack stops. Isolation drops all traffic from the source across every switch.

```
def isolate_source(self, src_ip, timeout=BLOCK_TIMEOUT):
    for d, dp in list(self.datapaths.items()):
        ofp, psr = dp.ofproto, dp.ofproto_parser
        m = psr.OFPMatch(eth_type=0x0800, ipv4_src=src_ip)      # any dst
        dp.send_msg(psr.OFPFlowMod(datapath=dp, cookie=REACTIVE_COOKIE,
            table_id=1, priority=110, match=m, instructions=[],  # [] = drop
            hard_timeout=timeout, flags=ofp.OFPFF_SEND_FLOW_REM))
```

Listing A.14: Source quarantine: a self-healing drop on every switch (REACTIVE_COOKIE = 0x00CA).

Anti-spoof GUARD (install_guard()). Table 0 binds each protected identity to its physical port and MAC; a frame carrying a protected source address that does not arrive on its bound port is dropped before it can reach policy. This is prevention, not detection.

```
for (d, port, mac, ip) in BINDINGS:               # bind identity to port+MAC
    if d == dp.id:
        add(200, OFPMatch(in_port=port, eth_type=0x0800,
                          eth_src=mac, ipv4_src=ip), GOTO)   # legit -> policy
if GUARD_ENABLED:
    for ip in PROTECTED_IPS:
        add(100, OFPMatch(eth_type=0x0800, ipv4_src=ip), [])  # forged src -> drop
add(0, OFPMatch(), GOTO)                           # default -> policy table
```

Listing A.15: Table 0 identity binding and anti-spoof drop.

Control-API authentication (_app()). Every state-changing endpoint requires a shared token; the read-only status endpoints and the Snort detection feed are excluded so the bridge is unchanged.

```
_CONTROL = ('/cars/defense','/cars/maintenance','/cars/reload','/cars/reload-a2',
            '/cars/block','/cars/unblock','/cars/restore')
if path in _CONTROL and environ.get('HTTP_X_CARS_TOKEN','') != self.api_token:
    return reply({"error": "unauthorized - X-CARS-Token required"}, '401 Unauthorized')
# /cars/respond (Snort bridge feed) is intentionally excluded
```

Listing A.16: Token gate on the control endpoints.

The supporting scripts

Detection bridge (snort_bridge.py). Snort raises the alerts; the bridge recovers the ICS operation from the alert text, measures its arrival rate, and posts the source, destination, operation and rate to the controller, which alone decides. The brain, not the sensor, holds the policy.

```
API = "http://10.10.10.1:8080/cars/respond"     # the brain decides; bridge only reports
mo = re.search(r'CARS-(MODBUS|S7)-(WRITE|READ|CONTROL|DIAG|PROGRAM|ILLEGAL)', line)
if mo: proto = mo.group(1); op = mo.group(2)     # recover the operation
key = (src, dst, op)                             # keep read vs write distinct
# rate of this (src,dst,op) measured, then POSTed with op+rate to API
```

Listing A.17: Operation recovery from the Snort alert, forwarded to the controller.

Flow-integrity self-check (`cars_flow_audit.py`). A trusted baseline of the immutable policy (Table 0 GUARD and the Table 1 A2 allowlist, cookie `0xa2`) is captured at a clean start; the only additions permitted at runtime are the reactive rules (cookie `0xca`). Anything else appearing on a switch is flagged as drift and surfaced into the controller's decision log.

```
# identity = bridge,table,priority,cookie,MATCH (not actions): a tampered action on a
# policy rule shows as "changed"; a bogus new rule as "extra".
def key_of(br, s): return "%s|t%s|p%s|c%s|%s" % (br, s[0], s[1], s[2], s[3])
# baseline = GUARD (t0) + A2 allowlist (t1, cookie 0xa2); the ONLY legitimate live
# additions carry the reactive cookie 0xca. Anything else later -> drift -> audit.
```

Listing A.18: Flow identity and the drift model.

Process-state remediation (`cars_remediation.py`). The novel maintain half of block-and-maintain: it detects sensor tampering by process anomaly (a level the bang-bang control law physically cannot produce) and writes the last-good value back over S7, holding the process while the source is quarantined.

```
last_good = SEED
while True:
    lvl = s7_read(c)                        # read Tank.Level over S7
    if tampered(lvl, prev):                 # a value the control law cannot produce
        s7_write(c, last_good)              # RESTORE last-good on tamper
        feed("RESTORED", level=round(lvl,1), last_good=round(last_good,1))
    else:
        last_good = lvl                      # healthy -> advance last-good
```

Listing A.19: Anomaly-triggered last-good restore over S7.

Latency harness (`cars2_mttm.py`). Times the reaction window on a single clock: start the attack, poll for the isolate to appear in the controller's block set, and record attack-frame-to-enforcement.

```
t0 = time.time()
# ... launch the attack ...
while not blocked() and time.time() - t0 < 15: time.sleep(0.02)  # poll for isolate
mttm = time.time() - t0                      # attack-frame -> enforcement
```

Listing A.20: Single-clock mean-time-to-mitigate measurement.

Attack tool (`cars_fdi_overflow.py`). The false-data injection used in Section 4.4: a compromised host opens an S7 session to the PLC and writes the output area at high rate, blinding the level sensor and jamming the discharge valve. Each write is a CONTROL operation to the output area, which CARS recovers and forbids.

```
c = snap7.client.Client(); c.connect(HOST, 0, 1)   # HOST = PLC 192.168.2.10
zero = bytearray(struct.pack('>f', 0.0))
while time.time() < end:
    c.write_area(PE, 0, 100, zero)   # blind LevelIn %ID100 = 0 (sensor OB30 trusts)
    c.write_area(PA, 0, 104, zero)   # jam DischargeValve %QD104 = 0 (output area 0x82)
```

Listing A.21: The FDI overflow injection (writes to the S7 output area).

Evaluation and proof harnesses

Accuracy harness (`cars_eval.py`). Drives the labelled case set of Section 4.2 through the deployed engine over `/cars/respond`, scores each answer against its intent label, and writes `cars_eval_matrix.csv`. It disarms for the decision-only pass so nothing is enforced on the live conduits, then re-arms.

```
def respond(src,dst,op,rate=0,dpid=1):           # one case -> the deployed decision path
    return post("/cars/respond",{"src":src,"dst":dst,"op":op,"proto":"S7","dpid":dpid,"rate":rate})
def verdict(label,resp):
    permit = resp in {"ALLOW","MONITOR","REFUSE"}
    if label in ("legit","flood_exempt"): return "TN" if permit else "FP"
    if label == "attack":                 return "TP" if not permit else "FN"
    return "grey"                                 # grey rows shown, not scored
post("/cars/defense",{"on":False},auth=True)      # decision-only pass; re-armed at the end
```

Listing A.22: Driving each case through the engine and scoring it against intent.

Criticality sweep (cars_criticality_proof.sh). Holds the actor and operation fixed and varies only the target's tier. Part 1 runs disarmed to show the bounded elevation as a pure decision; Part 2 runs armed to measure the tier-scaled quarantine ($30 + 15w$ seconds), healing the reactive rule after each fire.

```
fire(){ curl -s -XPOST $API/respond -H 'Content-Type: application/json' \
        -d "{\"src\":\"$2\",\"dst\":\"$3\",\"proto\":\"$4\",\"op\":\"$5\",\"dpid\":3}"; }
heal(){ for sw in ovsgw ovs1; do sudo ovs-ofctl -O OpenFlow13 --strict del-flows "$sw" "table=1,priority=110"; done; }
arm false     # PART 1 - decision side: bounded elevation, nothing enforced
arm true      # PART 2 - response side: real quarantine, hard_timeout = 30 + cw*15 s
```

Listing A.23: Fire a controlled case, and heal the tier-scaled reactive rule between fires.

Campaign library (cars_campaign_lib.sh). The shared helpers every campaign and pivot run sources: arm/disarm through the authenticated API, a cross-layer `snap` of the live state, and `capstart` which opens the three synchronised wire captures.

```
arm(){    curl -s -X POST $API/cars/defense -H "X-CARS-Token: $TOKEN" -d '{"on":true}';  }
disarm(){ curl -s -X POST $API/cars/defense -H "X-CARS-Token: $TOKEN" -d '{"on":false}'; }
capstart(){ sudo tcpdump -i "$PLCIF" -w "$d/plc1_wire.pcap" $filt &     # what reaches the PLC
            sudo tcpdump -i any     -w "$d/of_control.pcap" 'tcp port 6653' &  # the OpenFlow flow_mods
            sudo tcpdump -i snort0  -w "$d/dpi_mirror.pcap" & }          # what the detector sees
```

Listing A.24: Arm/disarm and the three-point capture used across the campaign.

Wire campaign (cars_wire_campaign.sh, cars_wire_campaign_disarmed.sh). Runs each attack vector against the armed system (and its disarmed twin), capturing the three wire points and timestamp-aligned state from every layer. The first vector is the control-plane disarm attempt.

```
sudo timeout 240 tcpdump -i "$PLCIF" -w "$D/plc1_wire.pcap" 'host 192.168.2.10 and tcp port 102' &
sudo timeout 240 tcpdump -i any     -w "$D/of_control.pcap" 'tcp port 6653' &
sudo timeout 240 tcpdump -i snort0  -w "$D/dpi_mirror.pcap" &
# V1 CONTROL-PLANE: unauth POST /cars/defense {on:false}  -> expect HTTP 401 + DENIED audit
```

Listing A.25: The three capture points and the control-plane vector.

Deep end-to-end trial (cars_e2e.sh). Drives the full response spectrum through the real autonomous chain and records, per scenario, the four independent proofs behind Table 4.4: the frame on the mirror, the Snort alert, the controller decision, and the datapath enforcement.

```
wirecount(){ tshark -r "$OUT/pcap/e2e_full.pcap" -Y "$1" | wc -l; }        # WIRE   : frames on the mirror
pkts(){ ovs-ofctl -O OpenFlow13 dump-flows "$1" table=1 | grep -m1 "$2" \
        | grep -oP 'n_packets=\K[0-9]+'; }                                  # OVS    : enforcement counter moved
# SNORT : delta on /var/log/snort/alert ;  CARS : tier+response in /cars/audit
```

Listing A.26: The four independent ground-truth layers per scenario.

Packet-drop proof (cars_packet_proof.sh). Captures the attacker's S7 frames at two synchronised points, the Snort mirror and the PLC's physical port, armed then disarmed, giving the drop-on-the-wire counts of Table 4.5.

```
capture(){   # tag seconds
  sudo timeout "$2" tcpdump -i snort0          "host $PLC1 and tcp port 102" -w "$D/$1_mirror.pcap" &  # DPI sees
  sudo timeout "$2" tcpdump -i enx9c69d331d874 "host $PLC1 and tcp port 102" -w "$D/$1_plc1.pcap"  & } # reaches PLC
arm true; heal; capture armed 15      # then: arm false; capture disarmed 15
```

Listing A.27: Two synchronised capture points: what the detector sees versus what reaches the PLC.

ICS operation battery (cars_ics_battery.sh). Fires the full ICS operation spectrum from one trusted operator (`.31`) at the Modbus unit (`.20`) and shows the three verdicts the one conduit draws, backing Table 4.6.

```
probe(){ restore; b=$(alast); ip netns exec opns "$@" >/dev/null 2>&1; wait_dec "$b"; }
# READ -> OPERATIONAL/ALLOW | WRITE -> SENSITIVE/THROTTLE | CONTROL,DIAG,PROGRAM,ILLEGAL -> FORBIDDEN/BLOCK
```

Listing A.28: Fire each operation on the same conduit and await the new decision.

Attack clients and the process model

S7 attack client (s7_write.py). The S7 tool used across the evaluation. Its output-area write is an actuation CARS classifies as CONTROL; the `--dbspoof` mode pins a false level (the sensor-spoof deception), and further modes cover monitoring, volumetric read floods and a PLC stop.

```
def w(v): c.write_area(PA, 0, 0, bytearray([v & 0xFF]))    # write Q0.x output image (area 0x82) = actuation
# -{}-dbspoof: pin a false level, deceiving both the control loop and the HMI:
payload = struct.pack('>f', a.spoofval)                    # S7 REAL = big-endian 32-bit float
c.db_write(a.db, a.offset, payload)                        # write-var 0x05 -> CARS classifies CONTROL
```

Listing A.29: The output-area write (actuation) and the sensor-spoof variant.

Modbus attack client (mb_attack.py). Crafts arbitrary Modbus/TCP function codes, including dangerous and illegal ones, to prove CARS decides on the operation and not the five-tuple. The function code sits at offset seven, exactly where the Snort DPI anchors.

```
def frame(fc, data=b'', tid=1, unit=1):
    pdu  = bytes([fc & 0xFF]) + data
    mbap = struct.pack('>HHHB', tid, 0x0000, len(pdu)+1, unit)   # proto-id 0x0000 @off2 ; FC @off7
    return mbap + pdu
ATTACKS = {'coil':0x05, 'diag':0x08, 'program':0x2B, 'illegal':0x64}  # CONTROL / DIAG / PROGRAM / ILLEGAL
```

Listing A.30: Crafting an arbitrary Modbus function code (FC at offset 7).

Process model (cars_process.py). The soft co-simulation of the water-tank bang-bang loop, driving the real Q0.3 relay on PLC1 over S7 from the authorised controller identity. It reasserts the actuator every cycle and reports readback-versus-intent mismatches, which are the moments an attacker's write momentarily reached the relay.

```
if   level <= LOW:  pump = True                # control law (eq.1)
elif level >= HIGH: pump = False
level += FILL if pump else -DRAIN              # dynamics (eq.2)
c.write_area(PA, 0, 0, bytearray([0x08 if pump else 0x00]))   # actuate + reassert Q0.3 every cycle
if (got != 0) != pump: bad += 1                # readback != intent = attacker interference reached the relay
```

Listing A.31: The bang-bang control law, the dynamics, and the interference health metric.

Gap mitigations: DPI hardening and the process guardian

DPI reassembly hardening (cars.conf, cars.rules). The single-packet S7 rules were defeated by TCP fragmentation (Section 4.5.1). Two changes closed it: stream reassembly on the ICS ports, and re-anchoring the operation match to the S7 job header instead of a fixed packet offset.

```
# cars.conf : reassemble the ICS conduits before inspection
preprocessor stream5_global: track_tcp yes, track_udp no, max_tcp 8192
preprocessor stream5_tcp:    policy linux, timeout 60, ports both 102 502
# cars.rules S7 Write-Var, before (fixed offset) -> after (PDU-relative):
#  before: content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|05|"; offset:17; depth:1;
#  after : content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|05|"; distance:8; within:9;
```

Listing A.32: Stream reassembly, and the offset-to-relative rule change.

Fragmentation harness (frag_s7_write.py). Opens a real allowlisted S7 session and sends a Write-Var either whole or split across TCP segments so the function byte falls in the second, the test behind the before/after of Section 4.5.1.

```
s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)   # Nagle off: each send() is its own segment
s.sendall(COTP_CR); s.recv(512); s.sendall(S7_SETUP); s.recv(512)   # real S7 session on an allowlisted conduit
if split <= 0: s.send(WRITEVAR)                           # whole  -> detected -> ISOLATE
else:          s.send(WRITEVAR[:split]); time.sleep(gap); s.send(WRITEVAR[split:])  # fragmented
```

Listing A.33: Sending the Write-Var whole or split at byte N (0x05 pushed into the second segment).

Low-rate insider tools (hmi_cmd.py, spoof_high.py). A single authorised HMI command, and a sustained high sensor spoof: the two residual insider actions of Section 4.9.

```
# hmi_cmd.py: one authorised write of the command merker %MB0 (bit 2 = HMI_Stop)
c.write_area(MK, 0, 0, bytearray([0x04]))                    # CARS classifies CONTROL -> ALLOW (trusted role)
# spoof_high.py: pin %ID100 high so Sim.Level (= LevelIn*20) reads ~full
c.write_area(PE, 0, 100, bytearray(struct.pack('>f', 4.5)))    # reported ~90 while the real tank drains
```

Listing A.34: One authorised HMI stop; and pinning the level sensor high.

Process guardian (cars_process_guardian.py). The defence-in-depth layer of Section 4.9: an independent, read-only monitor that adds the high-side envelope, symmetric rate and liveness checks the low-side remediation agent lacks, and raises alarms only. It never writes the PLC and never touches enforcement or the remediation agent.

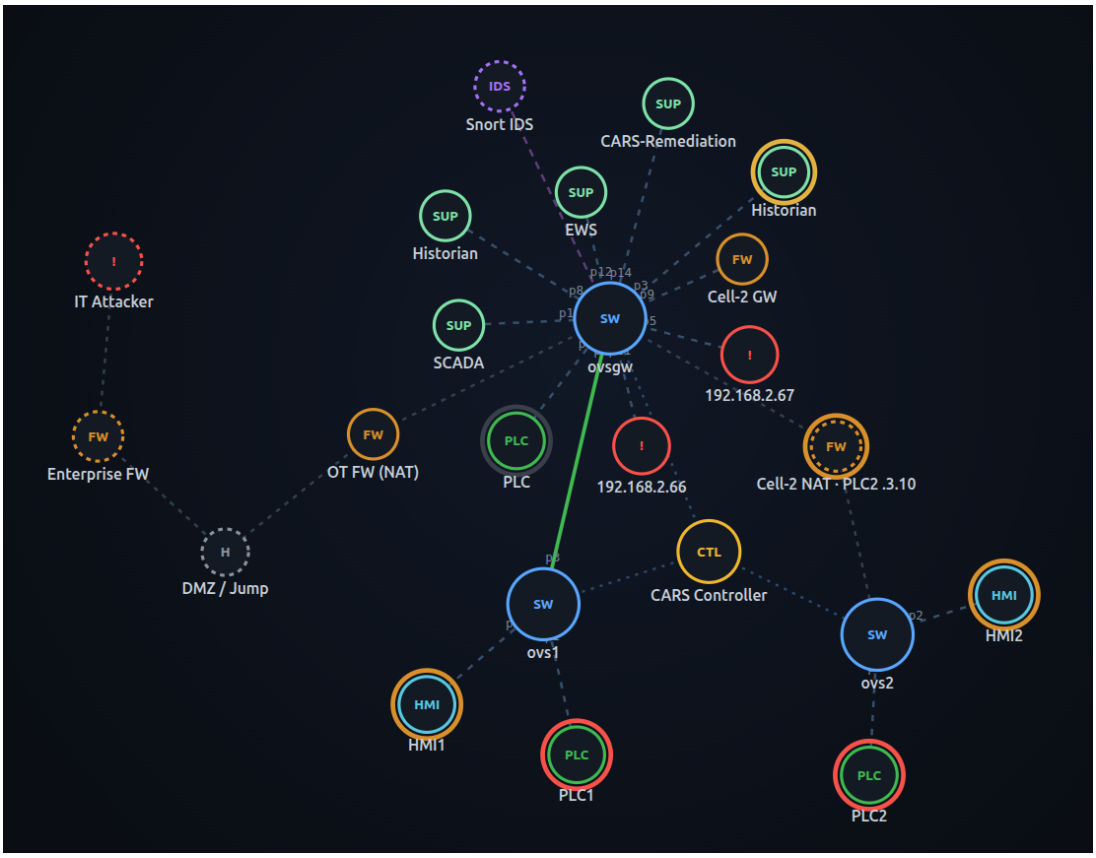

Figure A.10: The CARS dashboard's discovered topology. Switches (`ovsgw`, `ovs1`, `ovs2`) in blue, the supervisory hosts (EWS, SCADA, the historians, CARS-Remediation) and PLCs discovered live with their roles and criticality rings, the CARS controller and Snort IDS, the modelled north side (Enterprise firewall, DMZ/jump, OT NAT) and the Cell-2 NAT path to PLC2. Unregistered hosts (the on-segment attacker seams `192.168.2.66` and `192.168.2.67`) are flagged. Everything shown is polled live from the controller, not drawn by hand.

```
if lvl > CEIL:    alarm("HIGH_LEVEL", level=lvl, ceiling=CEIL)   # the excursion the low-side agent MISSES
if lvl < FLOORV:  alarm("LOW_LEVEL",  level=lvl, floor=FLOORV)
if prev is not None and abs(lvl - prev) > MAXDELTA:
    alarm("RATE", level=lvl, prev=prev)                          # a physically impossible per-poll jump
if abs(lvl - last_val) <= 0.6 and now - last_change > STALL_S:
    alarm("STALL", level=lvl)                                    # level frozen while it should cycle = stopped/held
```

Listing A.35: The high-side envelope, rate and liveness checks (alarm-only).

A.10 The CARS intelligence dashboard

The dashboard (`cars_dashboard.py`) is a live operator view of the defence rather than a static diagram. It is discovery-driven: a few times a second it polls the controller's REST interface (`/cars/status`, `/cars/hosts`, `/cars/ports`, `/cars/links`, `/cars/guard`, `/cars/audit`) and renders whatever the controller currently knows. The nodes (switches, hosts, the controller, and the modelled north side of IT, DMZ, firewall and Snort), together with their port bindings, link health, the anti-spoof GUARD drop counters and the engine's own decisions, are all pulled from the running system, so the picture is the live state and not a description of it. A separate pane reads the remediation agent's status and event feed from its local files, because that agent runs inside the OT namespace and cannot reach the control plane.

The centre of the view is the discovered network topology (Figure A.10); the right-hand column carries the live decision feed, filterable by criticality tier, response, operation class and enforcement mode. Each row shows the source and its discovered role, the destination and its criticality tier, the operation class, the criticality-elevated tier and the selected response, colour-coded by severity. It is this feed that makes

the engine's reasoning legible in real time: a supervisory read appears as an OPERATIONAL ALLOW, a remediation write as a RESTORE, and an out-of-role actuation as a FORBIDDEN ISOLATE.

Figure 4.4 in Section 4.4 is a capture of this feed during the armed false-data-injection run: the compromised supervisory host `192.168.2.31 (scada)` issuing S7 CONTROL to the CRITICAL PLC1, each attempt classed FORBIDDEN and met with an ENFORCED ISOLATE. Placed beside the datapath rule of Figure 4.5 and the wire capture of Section 4.5, it is the same attack seen at a third layer, the operator's, completing the cross-layer account: one event visible at once as a policy decision, an installed flow and a dropped packet.

A.11 Gap remediation evidence

This appendix collects the fresh before-and-after evidence for the late-stage gap remediations of Chapter 4, captured on 7 and 8 August 2026 on the armed testbed and returned to green after each run. The consolidated record with reproduction commands is the companion file `GAP_REMEDIATION_EVIDENCE.md`; the raw artefacts (controller-audit slices, the `cars.rules`/`cars.conf` diffs, the guardian log and the honeypot captures) are in `gap_evidence/`.

Gap A: DPI defeated by fragmentation, then hardened. A Write-Var split across two TCP segments evaded the fixed-offset single-packet rule; after enabling stream reassembly and re-anchoring the rules to the S7 job header it is recovered and isolated at more than one split point, with the legitimate loop still permitted.

```
BEFORE (single-packet rules) - fragmented at byte 14, armed:
  192.168.2.31(scada) -> 192.168.2.10(plc) TCP => ALLOW (operational)   [no CONTROL recovered; 0xca=0; write reached PLC]
  (same write sent WHOLE):
  FORBIDDEN 192.168.2.31(scada) -> 192.168.2.10(plc) S7 CONTROL => ISOLATE source 192.168.2.31 75s
AFTER (stream5 reassembly + PDU-relative rules) - same fragmented attack:
  --split 14: FORBIDDEN 192.168.2.31 ... S7 CONTROL => ISOLATE source 192.168.2.31 75s ; flow 0xca nw_src=192.168.2.31
  --split  9: FORBIDDEN 192.168.2.31 ... S7 CONTROL => ISOLATE
  legit loop: OPERATIONAL 192.168.2.55(ews) -> 192.168.2.10 READ/CONTROL => ALLOW   [no new false positive]
```

Listing A.36: Gap A: before (fragmented write evades) and after (caught), from the controller audit.

The fix itself is shown as the unified diff of the deployed Snort configuration and rules against the pre-hardening backup: Listing A.37 adds the TCP stream-reassembly preprocessor on the ICS ports, and Listing A.38 re-anchors every S7 operation rule from a fixed packet offset to a PDU-relative match on the S7 job header.

```
--- /home/msclab/cars_backup_2026-08-07_1715/cars.conf  2026-08-07 17:15:51.404757857 +0100
+++ /etc/snort/cars.conf        2026-08-07 18:57:12.948533211 +0100
@@ -2,4 +2,6 @@
 var EXTERNAL_NET any
 config checksum_mode: none
 config event_queue: max_queue 10 log 5 order_events content_length
+preprocessor stream5_global: track_tcp yes, track_udp no, max_tcp 8192
+preprocessor stream5_tcp: policy linux, timeout 60, ports both 102 502
 include /etc/snort/cars.rules
```

Listing A.37: Gap A: `cars.conf` diff, stream reassembly added on ports 102 and 502.

```
--- /home/msclab/cars_backup_2026-08-07_1715/cars.rules 2026-08-07 17:15:51.404705417 +0100
+++ /etc/snort/cars.rules       2026-08-07 19:12:56.120001291 +0100
@@ -1,4 +1,4 @@
-
alert icmp any any -> 192.168.2.10 any (msg:"CARS-SUSPECT ICMP PLC1"; sid:1000001; rev:5;)
+alert icmp any any -> 192.168.2.10 any (msg:"CARS-SUSPECT ICMP PLC1"; sid:1000001; rev:5;)
 alert icmp any any -> 192.168.2.9  any (msg:"CARS-SUSPECT ICMP HMI1"; sid:1000002; rev:2;)
 alert tcp  any any -> 192.168.2.10 any (msg:"CARS-SUSPECT TCP PLC1"; flags:S; sid:1000003; rev:1;)
 alert tcp  any any -> 192.168.2.9  any (msg:"CARS-SUSPECT TCP HMI1"; flags:S; sid:1000004; rev:1;)
@@ -12,22 +12,22 @@
 alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-CONTROL-coils"; content:"|00 00|"; offset:2; depth:2; content:"|0F|"; offset:7; depth:1; sid:1000022; rev:3;)
 alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-WRITE-regs";  content:"|00 00|"; offset:2; depth:2; content:"|10|"; offset:7; depth:1; sid:1000023; rev:3;)
 alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-READ-holding"; content:"|00 00|"; offset:2; depth:2; content:"|03|"; offset:7; depth:1; sid:1000024; rev:3;)
-# --- A3/P5 S7CommPlus session detection to the real PLC1 (.2.10:102) ---
-alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7COMMPLUS-session-PLC1"; content:"|03 00|"; depth:2; content:"|72|"; offset:7; depth:1; sid:1000040; rev:1;)
+# --- A3/P5 S7CommPlus session detection to the real PLC1 (.2.10:102) --- PDU-relative (Gap-2 reassembly fix)
+alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7COMMPLUS-session-PLC1"; content:"|03 00|"; content:"|72|"; distance:5; within:6; sid:1000040; rev:2;)
 # CC-51 broadened ICS intelligence: dangerous ops
 alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-DIAG-diagnostics"; content:"|00 00|"; offset:2; depth:2; content:"|08|"; offset:7; depth:1; sid:1000025; rev:1;)
 alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-PROGRAM-mei"; content:"|00 00|"; offset:2; depth:2; content:"|2B|"; offset:7; depth:1; sid:1000026; rev:1;)
 alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-ILLEGAL-fc"; content:"|00 00|"; offset:2; depth:2; byte_test:1,>,43,7; sid:1000027; rev:1;)
-# PD-3: classic S7comm Write-Var on real PLC1
-alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-CONTROL-write"; content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|05|"; offset:17; depth
       :1; sid:1000041; rev:1;)
+# PD-3: classic S7comm Write-Var on real PLC1 (PDU-relative)
+alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-CONTROL-write"; content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|05|"; distance:8; within:9;
       sid:1000041; rev:2;)
 # PD-7: classic S7comm PLC-control on real PLC1 - STOP (0x29) / Control-start (0x28) => DIAG (dangerous)
-alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-DIAG-stop";    content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|29|"; offset:17; depth:1;
        sid:1000042; rev:1;)
-alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-DIAG-control"; content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|28|"; offset:17; depth:1;
        sid:1000043; rev:1;)
+alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-DIAG-stop";    content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|29|"; distance:8; within:9; sid
       :1000042; rev:2;)
+alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-DIAG-control"; content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|28|"; distance:8; within:9; sid
       :1000043; rev:2;)
 # showcase: S7 read (func 0x04) => READ => OPERATIONAL (contrast vs write/stop)
-alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-READ-var"; content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|04|"; offset:17; depth:1; sid
       :1000044; rev:1;)
```

```
-# PLC2-2: Cell-2 target .3.10 S7 operation DPI
-alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-CONTROL-write"; content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|05|"; offset:17; depth
       :1; sid:1000045; rev:1;)
-alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-DIAG-stop";    content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|29|"; offset:17; depth:1;
        sid:1000046; rev:1;)
-alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-READ-var";     content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|04|"; offset:17; depth:1;
        sid:1000047; rev:1;)
-# N2 (audit 07-24): Cell-2 symmetry - Cell-1 has S7 control-start (0x28) DPI (sid 1000043) but Cell-2 lacked it. Add it.
-alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-DIAG-control"; content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|28|"; offset:17; depth:1;
        sid:1000048; rev:1;)
+alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-READ-var"; content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|04|"; distance:8; within:9; sid
       :1000044; rev:2;)
+# PLC2-2: Cell-2 target .3.10 S7 operation DPI (PDU-relative)
+alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-CONTROL-write"; content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|05|"; distance:8; within:9;
       sid:1000045; rev:2;)
+alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-DIAG-stop";    content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|29|"; distance:8; within:9; sid
       :1000046; rev:2;)
+alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-READ-var";     content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|04|"; distance:8; within:9; sid
       :1000047; rev:2;)
+# N2 Cell-2 symmetry: S7 control-start (0x28) DPI
+alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-DIAG-control"; content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|28|"; distance:8; within:9; sid
       :1000048; rev:2;)
```

Listing A.38: Gap A: `cars.rules` diff, S7 rules re-anchored PDU-relative (rev 1 to rev 2).

Gap B: trusted-insider process guardian. The high-rate overflow is cut by the volumetric overlay even from a trusted role; the read-only guardian then catches the low-rate denial and the high-side sensor spoof that the low-side remediation agent misses.

```
FLOOD BACKSTOP (trusted .45 driven at high rate):
  192.168.2.45(remediation) -> 192.168.2.10 S7 CONTROL => [FLOOD 51 ops/s] BLOCK conduit 75s  [FDI cut after 77 writes; tank held ~mid-70s]
DEMO 1 - low-rate denial (one authorised HMI_Stop from .45):
  audit:    192.168.2.45(remediation) -> 192.168.2.10 S7 CONTROL => ALLOW (operational)   [network correctly permits]
  guardian: [GUARDIAN] ALARM STALL {level: 40.4, frozen_s: 12.0}                          [process halt caught]
DEMO 2 - spoof-to-high (reported level pinned at 90, real tank drains):
  remediation status: level=90.0, restores unchanged                                      [low-side agent silent = blind]
  guardian: [GUARDIAN] ALARM HIGH_LEVEL {level: 90.0, ceiling: 78.0}  +  ALARM RATE       [high-side caught]
```

Listing A.39: Gap B: the flood backstop and the two guardian alarms.

The guardian writes each alarm to a read-only feed; the stall record from the fresh run is the raw line in Listing A.40. This is a crafted scenario that demonstrates the mechanism rather than broad operational resilience: the flood, envelope and rate thresholds are tuned to this tank and its loop rate, and would need re-tuning for a different plant, loop cadence or operator profile.

```
{"level": 40.4, "frozen_s": 12.0, "event": "PROCESS-ANOMALY", "kind": "STALL", "ts": 1786208299.0824745, "plc": "PLC1"}
```

Listing A.40: Gap B: the guardian alarm feed (`cars_guardian.jsonl`), stall record from the fresh run.

Gap C: flow-integrity poll window. A rule injected and deleted inside the ten-second poll evades the auditor; a persistent one is caught within one poll. The single traces below were then repeated thirty times each to quantify the window: the persistent case was caught on every trial at a median of 8.5 seconds, the two-second transient on only five of thirty (the runs whose life straddled a poll boundary). The per-trial values are in `gap_evidence/flowint/persistent.csv` and `transient.csv`.

```
baseline:                   {"ok": 1, "extra": 0}
2 s transient inject+delete:  t+7s ok:1 | t+14s ok:1 | t+21s ok:1     [MISSED - never flipped]
persistent inject (left):     {"ok": 0, "extra": 1}                   [CAUGHT within one poll]
after cleanup:                {"ok": 1, "extra": 0}
30-trial totals:  persistent 30/30 caught, median 8.5 s | transient 5/30 caught, 25/30 evaded
```

Listing A.41: Gap C: transient (mostly missed) versus persistent (always caught) flow-integrity status, with the thirty-trial totals.

The event-driven flow-monitor of Section 4.11 was then run against the identical thirty-trial transient test. It subscribes to the switch's Nicira flow-monitor (`ovs-ofctl monitor <br> watch:`) and, on any change to the policy tables that is not a cookie-`0xca` reactive rule, re-runs the deployed auditor's trusted check at once, so the sub-poll window closes without changing the verdict logic. Every one of the thirty two-second injections was caught, at a median of 0.27 seconds, against five of thirty for the ten-second poll; the per-trial values are in `gap_evidence/flowint/transient_eventdriven.csv`.

```
10 s poll (baseline):     transient  5/30 caught, 25/30 evaded (median 8.5 s when caught)
event-driven monitor:     transient 30/30 caught, median 0.27 s   [sub-poll gap closed]
mechanism: NXST_FLOW_MONITOR watch -> immediate trusted re-check on any non-0xca policy-table change
```

Listing A.42: Gap C remedy: event-driven monitor on the same transient test.

Gap 2: conduit block at a shared / NAT-collapsed identity. An IT attacker source-NATed onto the gateway identity `192.168.2.1` was originally cut by a source isolation, which quarantines every host behind the NAT (Section 4.7). The response selection was refined so a shared identity is cut at the conduit instead. Verified on the deployed engine: the same forbidden attempt from `192.168.2.1` now returns `BLOCK` and installs a conduit drop, while a true single host is still source-isolated.

```
gateway .1 (NAT-collapsed) -> CONTROL -> PLC1:  response=BLOCK
  flow: cookie=0xca priority=100,ip,nw_src=192.168.2.1,nw_dst=192.168.2.10 actions=drop   [conduit: only .1->PLC]
unknown .77 (true host)   -> CONTROL -> PLC1:  response=ISOLATE
  flow: cookie=0xca priority=110,ip,nw_src=192.168.2.77 actions=drop                      [source: whole host]
both land zero writes on the PLC; the gateway's non-PLC traffic is untouched by the conduit match.
```

Listing A.43: Gap 2: shared-identity conduit BLOCK versus single-host source ISOLATE, deployed engine.

Gap D: DEFLECT diverts to the honeypot. Forcing DEFLECT installs the redirect and the attacker's PLC-bound traffic arrives at the decoy while the real PLC sees nothing.

```
silent-drop baseline:  atkns(.66) ping 192.168.2.10 -> 100% loss                    [dropped; attacker hangs]
force DEFLECT:         response=DEFLECT  action="DEFLECT conduit -> honeypot 192.168.3.99 (deception, self-healing)"
redirect flow:         priority=105 nw_src=192.168.2.66 nw_dst=192.168.2.10 -> set ip_dst=192.168.3.99, goto t2 (n_packets=3)
honeypot capture:      IP 192.168.2.66 > 192.168.3.99: ICMP echo request            [attacker DIVERTED to decoy]
                       ARP, who-has 192.168.2.66 tell 192.168.3.99                   [decoy engaging]
```

Listing A.44: Gap D: silent-drop baseline, the redirect, and the decoy receiving the diverted traffic.

This is a proof of concept for the redirect path, not evidence of robustness. A determined adversary who notices the diversion, fingerprints the decoy or changes tactics is outside what this test exercises; sustaining a convincing interactive decoy is carried as future work in Section 5.3.
The plaintext control channel (G1), state-exhaustion resilience (G3) and the file-based detection bridge under heavy load (G4) are verified but framed as bounded limitations in Section 4.11 rather than tested, for the reasons given there.

A.12 Process-transparency and availability runs

This section holds the raw artefacts behind the process-transparency claim of Section 4.8 and the reconnection test of Section 4.11, captured on the armed testbed on 7 August 2026 and returned to green after each run.
Transparency: armed against disarmed under matched load. The two states were interleaved in short blocks over two hours so that any drift in the hardware-in-the-loop simulation fell on both equally. The per-second tank level, phase label, online flag and restore count are in `availability/interleaved.csv` (7,060 samples); the online-only summary is below. The means and spreads coincide and neither state left the control band, so arming the defence has no measurable effect on the process. As Section 4.8 states, this is strong evidence for the testbed and its hardware-in-the-loop tank rather than a universal guarantee for all industrial processes.

```
disarmed:  n=3528  mean=51.5%  sd=12.8  min=25.5  max=71.3   (band 20-78, no excursion)
armed:     n=3530  mean=50.7%  sd=13.2  min=25.6  max=71.3   (band 20-78, no excursion)
EWS/SCADA legitimate throughput unchanged across both phases.
```

Listing A.45: Transparency: online-only level statistics per phase, from `interleaved.csv`.

Block-and-maintain and self-heal, measured at the process. An allowlisted supervisory host launched a false-data injection at the CRITICAL PLC under the armed defence. The write was classified FORBIDDEN and the source isolated after nine packets with the criticality-scaled seventy-five-second timeout; the tank stayed within band for the whole block, the legitimate conduits kept flowing, and the restoration agent was never called. The rule then expired and the source returned to service with no operator action, the install-to-withdrawal spanning seventy-six seconds.

```
t+0.0s   FDI write from 192.168.2.31(scada) -> 192.168.2.10(plc, CRITICAL) => FORBIDDEN
t+~0s    ISOLATE source 192.168.2.31 (cookie 0xca, hard_timeout=75s, both switches)
block    tank level held within band; legit EWS/HMI conduits uninterrupted; restore agent NOT called
t+76s    rule expired (SEND_FLOW_REM); source restored, no operator action; table back to baseline
```

Listing A.46: Block-and-maintain under attack, then self-heal, from the armed run.

Reconnection jitter, two planes. Restarting the passive Snort sensor left the live conduit's inter-frame timing indistinguishable from baseline; forcing a controller reconnection by delete-and-re-add opened a roughly twenty-second window with multi-second forwarding stalls and dropped the synchronous S7 session. The per-second timeline (level, legitimate packets per second, controller-attached flag, event marks) is in `availability/jitter_timeline.csv` with the accompanying `hil.pcap`. As Section 4.11 sets out, that window largely reflects the heavy re-add and pipeline re-sync rather than a faithful crash against the `fail_mode=secure` switches, so it is reported as an upper bound and the clean crash-transparency figure is left as future work. This measurement should not be read as a crash-resilience proof or a production-availability guarantee: a fail-secure switch under a forced delete-and-re-add is not the same as a real controller crash, and the deployment implication is left deliberately cautious.

```
Snort restart (passive, out-of-band):  inter-frame mean 4.90 ms vs 4.91 ms baseline, worst gap 33 ms vs 42 ms  [TRANSPARENT]
Controller del/re-add (forced):         ~20 s window, worst forwarding gap 4.2 s, S7 session dropped, manual reconnect  [UPPER BOUND, not a faithful crash]
```

Listing A.47: Reconnection jitter: the two planes contrasted.